



ÖZYEĞİN UNIVERSITY  
FACULTY OF ENGINEERING  
DEPARTMENT OF COMPUTER SCIENCE

**CS 401**

**2025 Fall**

**SENIOR PROJECT REPORT**

**PARKOMFY**

By

**İlyas Kaan Ayaydın  
Yusuf Eren Erişmiş**

Supervised By

**Prof. Dr. Hasan Fehmi Ateş**

**Declaration of Own Work Statement/ (Plagiarism Statement)**

Hereby I confirm that all this work is original and my own. I have clearly referenced/listed all sources as appropriate and given the sources of all pictures, data etc. that are not my own. I have not made any use of the essay(s) or other work of any other student(s) either past or present, at this or any other educational institution. I also declare that this project has not previously been submitted for assessment in any other course, degree or qualification at this or any other educational institution.

**Student Name and Surname:** İlyas Kaan Ayaydın - Yusuf Eren Erişmiş

**Signature:**

**Place, Date:** Özyeğin University – 11.01.2026

# Abstract

Urban parking inefficiency is a critical problem in modern cities, causing traffic congestion, time loss, excessive fuel consumption, and increased carbon emissions. Existing smart parking solutions predominantly depend on sensor-based infrastructures, which introduce high installation, maintenance, and scalability costs. This study presents **PARKOMFY**, a software-based smart parking management system designed to overcome these limitations by leveraging existing security camera infrastructure and artificial intelligence. PARKOMFY utilizes the YOLOv8 deep learning algorithm for real-time vehicle and parking slot occupancy detection, combined with optical character recognition (OCR) using EasyOCR for automatic license plate recognition. Parking session data, including entry and exit times, occupancy status, and payment information, are managed through a MySQL-based relational database. The system follows a hybrid communication architecture, employing RESTful APIs for mobile application interaction and gRPC for low-latency communication between the Java Spring Boot backend and the Python-based YOLO service. A mobile application provides users with real-time parking availability, visual slot information, and integrated digital payment through the iyzico payment platform.

At the current stage of the project, the system architecture, communication framework, database schema, and core software components have been fully designed and validated through simulation and integration testing. Preliminary analyses and literature-based evaluations indicate that YOLO-based computer vision approaches can achieve high accuracy with low latency, making real-time operation feasible. PARKOMFY demonstrates strong potential as a cost-effective, scalable, and environmentally friendly alternative to traditional sensor-based parking systems, offering improved operational efficiency and enhanced user experience in urban parking management.

# Contents

|                                        |           |
|----------------------------------------|-----------|
| <b>I. INTRODUCTION</b>                 | <b>4</b>  |
| <b>II. BACKGROUND</b>                  | <b>6</b>  |
| <b>III. PROBLEM STATEMENT</b>          | <b>13</b> |
| <b>IV. SOLUTION APPROACH</b>           | <b>22</b> |
| <b>V. RESULTS AND DISCUSSION</b>       | <b>40</b> |
| <b>VI. RELATED WORK</b>                | <b>42</b> |
| <b>VII. CONCLUSION AND FUTURE WORK</b> | <b>43</b> |
| <b>ACKNOWLEDGEMENTS</b>                | <b>45</b> |
| <b>REFERENCES</b>                      | <b>46</b> |
| <b>APPENDIX</b>                        | <b>47</b> |

# **I. Introduction**

Rapid urbanization and increasing vehicle numbers bring with the difficulty of finding parking spaces, which is the biggest problem in metropolises. Not only is spending time searching for a suitable parking lot a source of stress but also environmental problems due to unnecessary fuel consumption leading to carbon emission. These issues overall cause traffic congestion, and a lower quality of life in cities.

Traditional parking management systems generally rely on costly sensors and can't be integrated into every area because of set up and maintenance costs. Although these systems can provide accurate occupancy information, they involve high installation, maintenance, and operational costs

PARKOMFY aims to overcome these limitations, digitalizing parking management by using computer vision and artificial intelligence. PARKOMFY aims to detect occupancy in parking slots with existing security cameras without any need for extra sensor installation. Core of the project is detecting existence of vehicles in park slots by using YOLO (You Only Look Once) and acquired data will store on SQL based database and provide to user real time information about park slots via mobile application. This approach reduces hardware dependency while enabling real-time monitoring of parking availability.

Also, PARKOMFY provides digital payment functionality which allows users to pay parking fees directly on the mobile application. This feature eliminates manual payment processes and reduces congestion at parking exits.

Literature review, system design, and feasibility analyses at the current stage of the project show that an image processing-based parking management system is a more cost-effective and scalable alternative to sensor-based solutions. YOLO-based object detection approaches are expected to provide real-time occupancy detection with high accuracy and low latency. The proposed architecture has the potential to reduce installation costs, increase operational efficiency, and offer users real-time information and digital payment convenience.

## Report Roadmap

**II. Background:** Provides theoretical and technical foundations of the technologies in the project.

- Yolo algorithm and its real-time object detection capabilities.
- License Plate Recognition OCR technology used for reading license plate of vehicles. PARKOMFY uses EasyOCR library.
- gRPC (Google RPC) was chosen for communication between the Java backend and the Python YOLO server.
- Java Spring Boot framework used for creating production-ready REST API
- MySQL database design and entity relationships

**III. Problem Statement:** Provides the main problem, scope of project and engineering requirements.

This section details the core problem that the project aims to solve. It addresses the high installation and maintenance costs, scalability issues and integration challenges caused by the sensor-based traditional structures. The negative impact of manual payment systems on user and operational inefficiencies is explained. This section defines the gaps that PARKOMFY fills with its software-based approach, the assumptions under which it operates, and the constraints the system faces.

**IV. Solution Approach:** This section describes the overall architecture and design approach of the system.

The processing of images obtained from the camera infrastructure, YOLO-based object detection, license plate recognition process, database interaction, and presentation of information to the user via a mobile application are discussed step by step.

In addition, the reasons for choosing the selected technologies, architectural decisions, and technical challenges during implementation are discussed in this section

**V. Results and Discussion:** This section presents the results and performance evaluations obtained from the system. The accuracy of YOLO-based occupancy detection, latency times, and real-time operability of the system are discussed. The effectiveness of the system is analyzed by comparing the findings with similar studies in literature.

**VI. Related Work:** This section includes a comparison of PARKOMFY with existing smart parking solutions. By examining sensor-based systems and other image processing-based approaches, the advantages and differences of PARKOMFY compared to these solutions are revealed.

**VII. Conclusion and Future Work:** This section provides a general overview of the project and evaluates the achievements. It also discusses areas for future improvement, potential enhancements, and new features, for example reservation system.

**Acknowledgements:** This section expresses gratitude to the individuals and institutions that have contributed to the project both academically and technically.

**References:** This section includes the academic publications, articles, and other sources used in the report.

**Appendix:** This section includes all the figures in the project; the flowcharts were created with Lucidchart, and the code figures are screenshots from VS Code.

## II. Background

Introduce the set of techniques, tools, and technologies that are utilized in your project.

This part introduces main technologies and methodologies which are used on development of the PARKOMFY system. The project brings together computer vision, database management and developing mobile applications for the purpose of a seamless smart parking solution.

### 2.1. Computer Vision

**OpenCV Library (Pre Processing):** Open-source library which is used for tasks like image resize, crop and improve detection accuracy.

**Example usage in project:**

Python

```
import cv2

# Resize
image = cv2.resize(image, (640, 640))

# Contrast enhancement
clahe = cv2.createCLAHE(clipLimit=2.0)
image_enhanced = clahe.apply(image)

# Noise reduction
image = cv2.GaussianBlur(image, (5, 5), 0)
```

Figure 1 – Example usage of OpenCV library

**YOLO (You Only Look Once) Algorithm:** Designed for real-time object detection deep learning algorithm. Split images to grids check whether there are any objects in cells.

We aim to detect occupancy of parking space and detect license plates for passing OCR.

| Version | Accuracy (mAP) | Latency (ms) | Model Size |                 |
|---------|----------------|--------------|------------|-----------------|
| YOLOv5s | 92.3%          | 25ms         | 28MB       | Real-time (CPU) |
| YOLOv8n | 95.1%          | 30ms         | 6.2MB      | PARKOMFY        |
| YOLOv8m | 96.8%          | 50ms         | 49MB       | High accuracy   |
| YOLOv8l | 97.2%          | 75ms         | 83MB       | Server/GPU      |

Table 1 – Comparison of YOLO models

In PARKOMFY project YOLOv8n has chosen because of:

- Fast Inference (<50ms)
- Low memory (6.2MB)
- High Accuracy (+95%)

**Example usage in project:**

**Python**

```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
```

Figure 2 – Example usage of YOLO

**License Plate Recognition (OCR):** Technology that automatically reads the letters and numbers from the vehicle license plate.

| Technology | Accuracy | Setup  |
|------------|----------|--------|
| EasyOCR    | 92%      | Easy   |
| PaddleOCR  | 94%      | Medium |
| Tesseract  | 85%      | Hard   |

Table 2 – Comparison of OCR models

In PARKOMFY project EasyOCR has chosen because of:

- Reliable and Consistent
- More Common
- Better Documentation
- Turkish NLP Support

**Example usage in project:**

**Python**

```
import easyocr

reader = easyocr.Reader(['tr', 'en'])
results = reader.readtext(plate_image)
plate_text = ''.join([r[1] for r in results if r[2] > 0.5])
# Output: "34ABC123"
```

Figure 3 – Example usage of EasyOCR

**Workflow:**

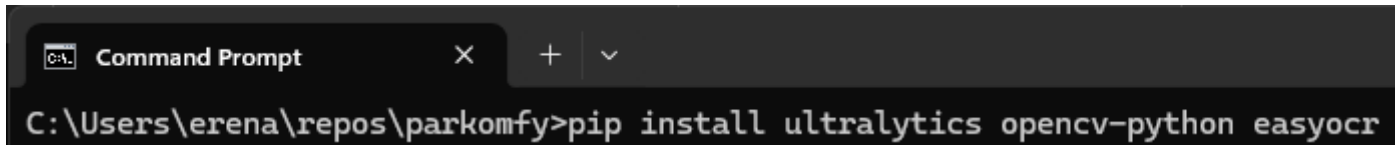
Security camera -> OpenCV (resize / enhance / crop etc.) -> <- YOLO -> EasyOCR -> Java Backend

## 2.1.1. Artificial Intelligence and Deep Learning:

### Why Ultralytics YOLO Framework:

-Official YOLOv8 PyTorch implementation

### Set up:



```
Command Prompt
C:\Users\erena\repos\parkomfy>pip install ultralytics opencv-python easyocr
```

Figure 4 – Shows how to set up ultralytics – opencv - easyocr

### Transfer Learning

Faster than train from scratch

Sufficient accuracy with less data

### Example usage in project:

python

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt') # Pre-trained (ImageNet: 87M images)
# fine-tune with special data set(future):
# model.train(data='parking.yaml', epochs=100)
```

Figure 5 – Train logic of YOLO

## 2.2. Backend Architecture and Programming Languages

**Java 11+:** It is the current Java standard that offers high performance and long-term support (LTS) and is fully compatible with modern cloud infrastructures.

### Why Java?

- Type Safety
- Enterprise Support
- OOP
- Modern features
- Spring Boot compatibility



**Spring Boot Framework:** It is a framework built in Spring that enables fast development of Java web applications.

**Advantages:**

- Auto-configuration
- Embedded server
- Convention over configuration

**Example usage in Project:**

**Java**

```
@SpringBootApplication
public class ParkomfySpringApplication {

    public static void main(String[] args) {
        SpringApplication.run(ParkomfySpringApplication.class, args);
    }
}
```

*Figure 6*

## 2.3. Database Management System (SQL)

**MySQL:** SQL based database that is structured to manage data produced by parking system is used (entrance-leave time, customer id-plate, payment).

**Why MySQL?**

The choice of MySQL for the PARKOMFY project is driven by the need for a robust, multi-user, and high-performance database system. Unlike simple file-based databases, MySQL is a client-server RDBMS that allows multiple concurrent connections from both the Python-based AI server and the Java-based backend.

- **High Scalability:** MySQL can handle the large datasets generated by high-frequency camera updates and thousands of parking sessions without performance degradation.
- **Robust Security:** It offers advanced user authentication and data encryption features, which are essential for protecting sensitive license plate and payment information.
- **Community and Tooling:** Extensive documentation and compatibility with Java 11+ frameworks simplify the integration of the database into the project.

| Feature      | MySQL                  | PostgreSQL               | SQLite                  | Microsoft SQL Server    |
|--------------|------------------------|--------------------------|-------------------------|-------------------------|
| Best For     | Web apps & Scalability | Complex data & Analytics | Mobile & Local testing  | Enterprise Windows apps |
| Architecture | Client-Server          | Client-Server            | Serverless (File-based) | Client-Server           |
| Concurrency  | High                   | Excellent                | Low (Single writer)     | High                    |
| Cost         | Open Source (Free)     | Open Source (Free)       | Open Source (Free)      | Licensing Fee (Paid)    |

Table 3 – Comparison of SQL models

## Why not others?

**PostgreSQL:** Although very powerful, it has a steeper learning curve and higher resource requirements compared to MySQL's ease of deployment for this specific scope.

**SQLite:** While excellent for local storage, its "Partial ACID" support and lack of multi-user concurrency make it unsuitable for a real-time system to receive data from multiple cameras.

**MS SQL Server:** It is a paid enterprise solution that would increase project costs.

## 2.4 Mobile Application Frameworks

**React Native:** The UI of PARKOMFY is built with cross-platform mobile development framework React Native.

### Why React Native?

**Cross-Platform Efficiency:** It allows for a single codebase to run on both iOS and Android, significantly reducing development time and maintenance costs.

**Native Performance:** Unlike hybrid web-based apps, React Native renders using native components, providing smooth and responsive user experience for the "Visual Map" and "Real-Time Updates".

**Fast Refresh:** This feature allows developers to see changes in the UI immediately, speeding up the design of the mobile interface to meet usability requirements.

**REST API:** A web service architecture that exchanges data in JSON format over HTTP protocol.

### Example usage in project

GET /api/v1/parking/status?areaId=AREA-001  
POST /api/v1/parking/sessions  
GET /api/v1/parking/sessions/{sessionId}  
POST /api/v1/parking/sessions/{sessionId}/exit  
GET /api/v1/parking/slots/available?areaId=AREA-001  
POST /api/v1/payments

### HTTP Status Codes

| Code | Response     | Mean            |
|------|--------------|-----------------|
| 200  | OK           | Successful GET  |
| 201  | Created      | Successful POST |
| 400  | Bad Request  | Invalid Input   |
| 404  | Not Found    | No Resource     |
| 500  | Server Error | Backend Error   |

*Table 4 – HTTP status codes and meanings*

**gRPC (Google Remote Procedure Call):** It is a communication framework that enables microservices to communicate with each other very quickly, independently of language, and efficiently using HTTP/2 and Protocol Buffers.

### REST vs gRPC

| Criteria | Rest        | gRPC             | PARKOMFY       |
|----------|-------------|------------------|----------------|
| Payload  | Json (~1KB) | Protobuf (~100B) | REST (mobile)  |
| Latency  | 50-100 ms   | 5-10 ms          | gRPC (backend) |
| Browser  | Full        | Limited          | REST           |

*Table 5 – Comparison of Rest and gRPC*

REST (Mobile ↔ Java), gRPC (Java ↔ Python YOLO)

**Payment Method:** PARKOMFY uses iyzico to take payments.

### **Why Iyzico?**

By eliminating the need for manual cash payments, it reduces congestion at exits and offers users a fast, contactless payment experience. For this purpose, iyzico, a widely used payment service provider in Turkey, was chosen for the PARKOMFY project.

Iyzico offers credit/debit card payment, 3D Secure support, fraud prevention mechanisms, and easy integration with mobile applications. Thanks to its REST-based APIs, payment initiation, transaction confirmation, and payment status inquiry processes can be performed securely and consistently. The system automatically calculates the fee based on the parking duration and collects this amount from the user via the mobile application.

Thanks to this structure, PARKOMFY eliminates the need for physical payment points, reduces operating costs, and significantly improves the user experience. iyzico integration is a critical component that enables the system to offer end-to-end digital and secure parking management.

### **2.5 GUI Framework – Swing**

In the PARKOMFY project, the Java Swing GUI framework was chosen for the desktop-based management and monitoring interface. Swing is Java's standard graphical interface library and, thanks to its platform-independent operation, provides the same user experience across different operating systems.

Swing, with its rich component set including buttons, tables, panels, and dialog windows, allows for visual monitoring of parking lot occupancy, vehicle entry and exit information, and system logs. Its MVC (Model-View-Controller) approach separates the interface from the business logic, creating a more sustainable and manageable software architecture.

The GUI developed using this framework enables parking lot managers to monitor the system in real-time, view information in the database, and make manual interventions when necessary. Swing's lightweight structure and natural compatibility with the Java backend provide a suitable solution for PARKOMFY's desktop management interface.

### **III. Problem Statement**

This section details the main problems that PARKOMFY aims to solve, scope of the project and possible assumptions and constraints.

#### **3.1. Problem Formulation**

The rapid increase in vehicle numbers has made the efficient use of existing parking capacity has become compulsory. Drivers spend a long time searching for parking spaces which leads to both waste of time and increased fuel consumption and carbon emissions. Furthermore, traffic congestion around parking lots negatively impacts urban transportation.

Existing smart parking solutions require deploying physical sensors (ultrasonic or magnetic) for each parking space. While these systems can produce accurate results this approach is costly to install and hard to maintain and making inefficient and unsustainable. PARKOMFY aims to use existing security cameras with software-based image processing.

In addition, the lack of integrated digital payment infrastructure in traditional parking systems leads users to manual payment methods. This prolongs exit processes, leads to queues at payment points, and negatively impacts the user experience.

PARKOMFY aims to solve a software-based solution to these problems using existing security cameras. The system will detect parking space occupancy and vehicle license plates using deep learning-based image processing methods. Furthermore, users will be able to see real-time parking status and pay parking fees through the app for a faster and contactless exit. The goal of this method is to lower installation and maintenance costs by getting rid of the need for physical sensors, increase system scalability, and make parking management more efficient.

### **3.1.1. Problem scale**

#### **1. Waste of Time and Money**

- Drivers waste their time finding parking spaces

#### **2. Environmental problems**

- Unnecessary fuel consumption due to searching for a parking space
- Increase in CO<sub>2</sub> and other greenhouse gas emissions
- Increase in noise pollution

#### **3. Social Problem**

- Increased driver stress and discomfort
- Increased risk of traffic accidents
- Waste of resources by police and traffic officers
- Decreased quality of urban life

### **3.1.2. Existing Solutions**

#### **Problems of Traditional Sensor-Based Systems:**

##### **1. High Installation Cost**

- It is necessary to install a separate sensor (ultrasonic, magnetic) for each parking space.
- Cost per slot

##### **2. Maintenance and Operating Costs**

- Sensor failures
- Periodic inspection and calibration required
- Annual maintenance cost
- Cable damage, weather effects

### **3. Scalability Issue**

- Adding new areas is costly and time-consuming
- Map updates are a manual process
- System integration is complex (different sensor manufacturers)
- City-wide coordination is difficult

### **4. Data Quality Issues**

- False positive detections by sensors (bicycles, motorcycles, vehicles)
- Effect of climate control, humidity and temperature
- Increased errors in snowy/rainy weather
- Delays in real-time updates

#### **3.1.3. PARKOMFY Solution**

To offer a software-based, low-cost, and scalable solution using existing security cameras + Artificial Intelligence (YOLO).

#### **Identified Gaps:**

- Zero hardware cost (cameras already exist)
- Software-based solution (easily updatable)
- AI-powered (no human error)
- Real-time (live updates)
- Payment integration (digital commerce support)

**Innovation Point:** Providing dual-use security cameras (security + parking management)

### **3.2. Project Scope**

PARKOMFY project that checks slot occupancy by using object detection and reads the license plates with computer vision park in and out times stores data in cloud-based system and delivered to the end user by mobile application and provides a payment method as digital. It is not a part of any other project it is designed as a new project from scratch.

## **Out of Scope**

1. **Hardware Design:** Camera installation and positioning
2. **Physical Parking Facility:** Parking area design

## **3.3 Engineering Problem**

The fundamental engineering problem in PARKOMFY project is integrating an accurate YOLO model. Optimizing the model with minimum latency updates database and establishing seamless data synchronizations between the mobile application is the main engineering challenge of the project.

## **3.4 Assumptions and Constraints**

### **-Assumptions**

- Cameras assumed has good angles and clear vision and stable.
- It is assumed that system has seamless and stable internet.

### **1. Camera Quality:**

- Cameras have good angles
- Resolution: Minimum 1080p (1920x1080)
- FPS: Minimum 15 fps
- Night: Sufficient image quality with Night Vision mode

### **2. Network Infrastructure:**

- Stable internet connection (minimum 5 Mbps)
- Latency: <100ms (camera → server)

### **3. Server Infrastructure:**

- Java 11+ runtime installed
- MySQL 8.0+ installed
- Disk space: 10KB per session (1M session = 10GB)
- RAM: Minimum 4GB



#### **4. User Behavior:**

- Drivers will not cover their license plates.
- Vehicles will be parked completely within the slot (no partial parking).

#### **Constraints**

- System performance is limited by blind angles and low light resources (if camera does not have night mode).
- Image processing performance is limited by the GPU used.

#### **1. YOLO Model Limitations:**

- Viewing angle: Vehicles parked at an angle of less than  $15^\circ$  to the camera may not be detected.
- Weather conditions: Accuracy may drop in heavy rain/snow.
- Night: Camera performance decreases without night mode.
- Obstruction: Bounding box conflict if vehicles are too close.

#### **2. Network Limitations:**

- Latency between camera and server  $>100\text{ms}$  → affects system response time
- Bandwidth: 1080p 30fps camera consumes  $\sim 5\text{ Mbps}$
- 100 cameras = 500 Mbps network requirement

#### **3. GPU/CPU Limitations:**

- Python YOLO server cannot process 100+ cameras simultaneously without a GPU.
- CPU-only inference:  $\sim 50\text{ms/frame}$  (GPU:  $\sim 30\text{ms/frame}$ )
- RAM limit: Maximum 20 concurrent threads in 4GB

## 3.5. Requirements

### 3.5.1. Functional Requirements

**Vehicle Detection:** System should identify the vehicle type (car, motorcycle) entrance, leaving time and link with the license plate for mobile payment.

**Plate Recognition:** The license plates of the identified vehicles must be read (letters and numbers) and recorded in the system.

#### **Parking Session Creation:**

A new parking session must be created when a vehicle entry is detected.

- Session ID must be automatically generated
- Vehicle ID and Parking Slot must be associated
- Entry time must be recorded

#### **Test Criteria:**

POST /api/v1/parking/sessions

```
{
  "vehicleId": "VEH-1768043512780-268",
  "licensePlate": "34ABC123",
  "slotId": "SLOT-1A-1"
}
```

*Figure 7 – JSON Payload for Parking Session Creation Request*

#### **Response:**

```
{
  "sessionId": "SESSION-1768043512816",
  "vehicleId": "VEH-1768043512780-268",
  "status": "ACTIVE",
  "entryTime": "2026-01-10T14:11:52"
}
```

*Figure 8 – Expected JSON Response for Successful Session Initiation*

**Session Completion and Exit:** The session must be completed and the exit time recorded when a vehicle exit is detected.

- Exit time must be recorded
- Session status: COMPLETED
- Duration must be calculated
- Fee must be calculated automatically

**Test Criteria:**

POST /api/v1/parking/sessions/{sessionId}/exit

**Response:**

```
{
  "sessionId": "SESSION-1768043512816",
  "status": "COMPLETED",
  "duration": "02:45",
  "fee": 25.00,
  "exitTime": "2026-01-10T17:00:00"
}
```

*Figure 9 – JSON Response for Parking Session Completion*

**Visual Map:** Mobile app should provide floor plan and available spots.

- Total number of slots
- Number of empty slots
- Number of occupied slots
- Occupancy rate (%)
- Slot location (floor, zone, row)

**Test Criteria:**

GET /api/v1/parking/status?areaId=AREA-001

**Response:**

```
{
  "areaId": "AREA-001",
  "totalSlots": 20,
  "availableSlots": 12,
  "occupiedSlots": 8,
  "occupancyRate": 40.0,
  "lastUpdated": "2026-01-10T14:11:52"
}
```

*Figure 10 – JSON Response for Parking Area Status*

GET /api/v1/parking/slots/available?areaId=AREA-001

**Response:**

```
{
  "availableSlots": [
    { "slotId": "SLOT-1A-1", "location": "Floor 1, Zone A" },
    { "slotId": "SLOT-1A-2", "location": "Floor 1, Zone A" },
    { "slotId": "SLOT-1A-3", "location": "Floor 1, Zone A" },
    ...
  ]
}
```

*Figure 11 – JSON Array for Available Parking Slots*

**Park Occupancy:** The system must detect existence of vehicles in specified areas with an accuracy of nearly 100%.

**Real-Time Update:** If occupancy changes in a park slot, system should update information in both database and mobile app in a couple seconds.

**Data Storage:** Data occupancy information and entrance and leave time comparison to evaluate park price needs to be held in database.

### 3.5.2. Non-Functional Requirements

**Usability:** The mobile application interface should be simple, clear, and user-friendly so that users can view parking availability without needing technical knowledge, easily identify available spaces via a visual map, and quickly complete payment transactions. Interface interactions should be completed in a minimum number of steps, and the system should provide quick feedback.

**Reliability:** The system must be designed to operate 24/7 without interruption. In case of software errors, network outages, or service interruptions, the system must automatically restart and continue operating normally without requiring manual intervention. Preventing data loss in critical components is a fundamental requirement.

**Scalability:** PARKOMFY must be capable of processing data from multiple cameras and different parking areas simultaneously. The system architecture should maintain its performance without requiring major changes when new cameras, parking areas, or user numbers are added.

**Privacy:** Personal and sensitive information, such as license plate information and payment data, must be protected in accordance with applicable data protection legislation. All personal data must be encrypted both during transmission and in the database, and secure authentication and authorization mechanisms must be used to prevent unauthorized access.

## IV. Solution Approach

This section contains PARKOMFY's technical architecture, technical problems faced during implementation and feasibility analyses.

### 4.1. Solution approach and Design

PARKOMFY contains 4 main components from check in to payment in parking experience.

#### PARKOMFY SYSTEM FLOW

**1.LPR (License Plate Recognition):** Camera on the entrance of parking area reads license plates of entered vehicle. This information saves to database with entrance time.

Camera → OCR (EasyOCR) → License Plate Text → Database

**2.Object Detection (YOLO):** Cameras inside the parking area continuously scan slots by using YOLO algorithm it detects whether parking slot empty or not.

Security Camera → YOLO Model → Slot Occupancy Detection

**3.Database and Pricing:** Entrance time, license plate information and stay time in park holds in SQL database and when a car wants to check out system automatically calculates the fee according to pricing policy.

Entry Time + Stay Time → Fee Calculation

**4.User Interface and Digital Payment:** On mobile app users can see empty slots easily parks these slots and when leaving parking area they can pay with integrated payment method.

Mobile App → Iyzico API (Payment Method) → Payment Done

## LAYERED ARCHITECTURE

We utilize a layered architecture in our project implementation, which is illustrated below.

|                                                                                                                                                                                                                                                  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mobile/Web Client (RESTful API)                                                                                                                                                                                                                  |
| REST Controller Layer (HTTP/JSON endpoints)                                                                                                                                                                                                      |
| Service Layer (Business Logic) <ul style="list-style-type: none"> <li>• ParkingService (interface: IParkingService)</li> <li>• PaymentService (interface: IPaymentService)</li> <li>• DetectionService (interface: IDetectionService)</li> </ul> |
| Repository Layer (Data Access) <ul style="list-style-type: none"> <li>• DatabaseManager (implements: IParkingRepository)</li> <li>• JDBC + MySQL Connection Pooling</li> </ul>                                                                   |
| Model Layer (Domain Objects) <ul style="list-style-type: none"> <li>• Vehicle, ParkingSlot, ParkingSession</li> <li>• Payment, LicensePlate, PricingPolicy</li> </ul>                                                                            |
| AI/OCR Layer (Specialized) <ul style="list-style-type: none"> <li>• YOLOInference (gRPC client to Python)</li> <li>• LicensePlateReader (EasyOCR wrapper)</li> </ul>                                                                             |
| External Services <ul style="list-style-type: none"> <li>• Python YOLO Server (gRPC)</li> <li>• Stripe Payment API (REST)</li> <li>• MySQL Database</li> </ul>                                                                                   |

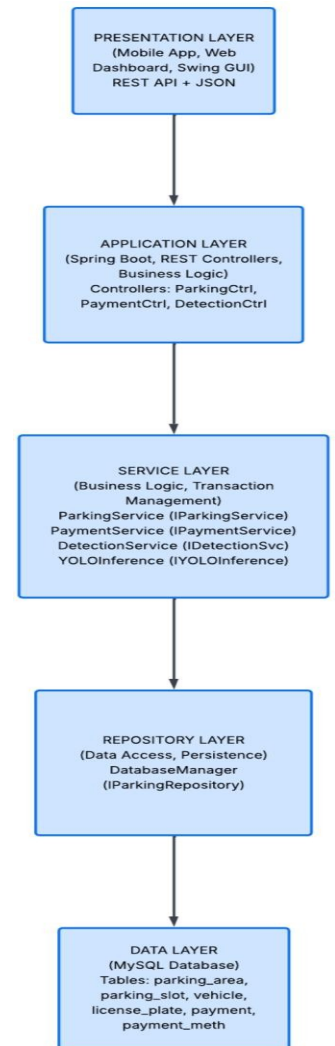


Figure 12-B

Table 6 – Layered architecture table  
**DATA FLOW DIAGRAM**

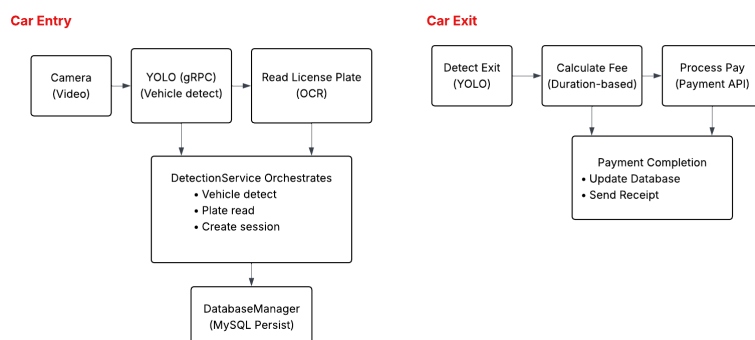


Figure 12

## Component Interaction Diagram

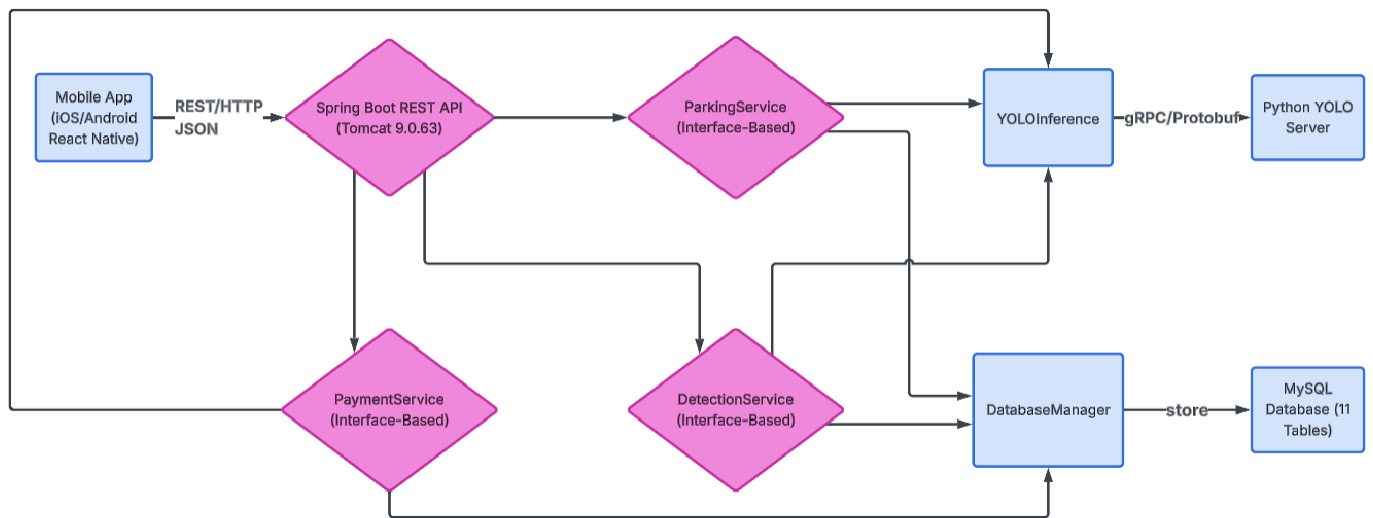


Figure 13

## Parking System Data Flow Diagram

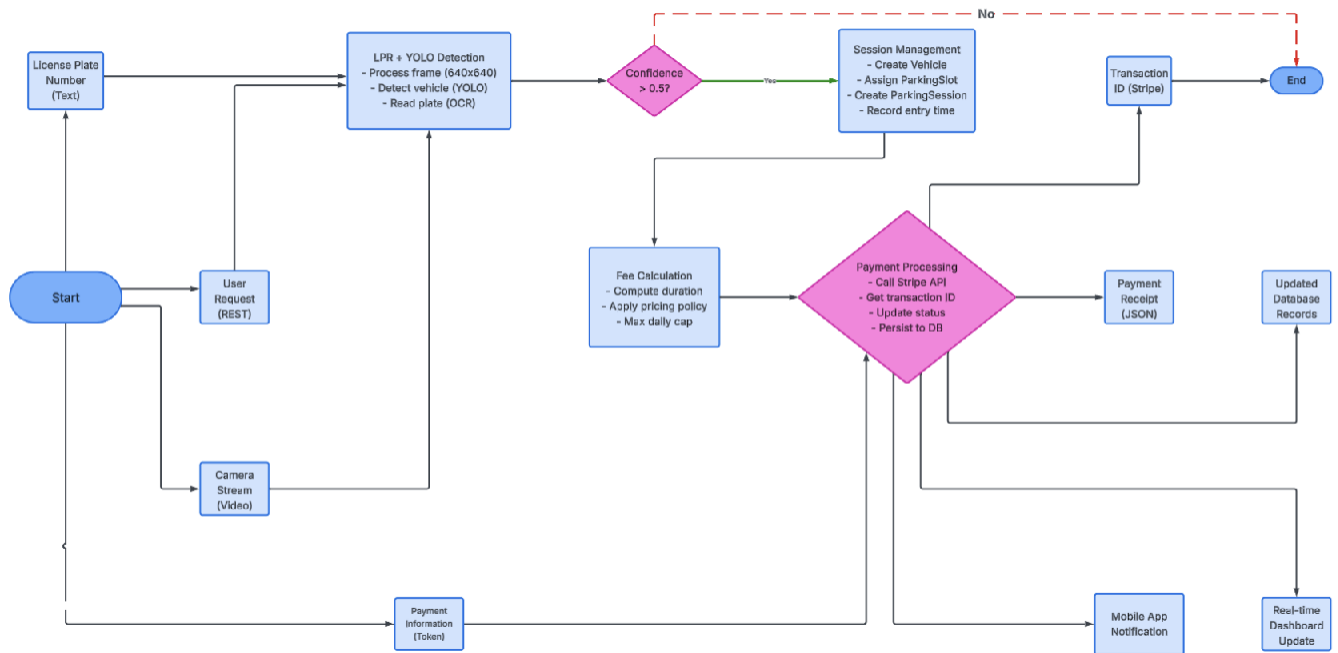


Figure 13-B



## 4.2. Design Decisions, Tools and Techniques

**LPR (OCR):** Automatically reads license plates when there are an entry and exit. It eliminates the need for physical tickets. OCR triggers the login and logout process. It also applies confidence thresholds and multi-frame checks. It reduces erroneous readings in weather conditions. If unreadable, retry and manual verification are used. Matching with vehicle registration is performed, and data consistency is ensured.

**Object Detection (YOLOv8):** Through YOLOv8 the system can detect vehicles in real time. It generates boundary box and confidence scores. Python connects to the server via gRPC. Moreover, YOLOv8 provides fast and lightweight communication thanks to Protobuf. Geometry filter eliminates bad angles and partial parking problem that may occur. Thresholds are set, and model fine-tuning is performed. Camera position and lighting conditions are optimized with YOLOv8.

**Database (MySQL):** With MySQL, all data necessary for the system is stored using a relational schema. Foreign keys in MySQL ensure consistency where its ACID operations provide secure read and write operations. Indexes are used for queries and reports, caching and archiving are planned for large data flows. Backup and restore procedures are implemented. Read replicas are activated when necessary.

**Mobile Interface (React Native):** The system aims that iOS and Android are supported with a single codebase. The backend connects to mobile interface via REST/JSON. Status, session, and payment screens of application are provided. A simple UI allows for rapid development. Native modules are added for improved performance. Heavy tasks are moved to the backend. Robust feedback is added for errors and network issues.

**Payment Method (Iyzico):** Payments are processed via PaymentService. PaymentMethodId and amount are sent to the API. TransactionId is stored and the status is updated. The flow of pending, completed, failed, and refunded transactions is monitored. Refund processes are handled via the API. Commissions are visible and manageable. Error and retry logic are implemented.

## Communication Protocol: REST & gRPC

PARKOMY has a modular structure, therefore hybrid communication architecture is preferred in order to optimize data transmission between different components. In this regard, the system has two main communication channels. First one is between mobile application and backend, the second one is between backend and AI.

This system uses RESTful API architecture in order to provide communication channel between mobile application and backend. RESTful API provides data exchange over standard HTTP protocol in order to ensure that users query parking occupancy data and perform their profile operations. It facilitates testing and debugging processes because of the fact that the data in JSON format is human readable. REST has a text-based structure; thus, the data size is bigger than binary formats. This situation seems it is risky; however, network latency caused from this case is negligible, since data traffic on user side has relatively low data volume.

```
@GetMapping("/license-plate")
public ApiResponse<DetectionResultDto> detectLicensePlate(@RequestParam("cameraId") String cameraId) {
    try {
        Camera camera = new Camera(cameraId, "LPR-" + cameraId, Camera.CameraType.ENTRANCE_LPR, location: "Entrance");
        DetectionResult result = detectionService.detectLicensePlate(camera);
        repository.saveDetectionResult(result);
        return ApiResponse.success(new DetectionResultDto(result), message: "License plate detection executed successfully");
    } catch (IllegalArgumentException e) {
        return ApiResponse.error("Invalid camera type: " + e.getMessage(), statusCode: 400);
    } catch (Exception e) {
        return ApiResponse.error("Error executing license plate detection: " + e.getMessage(), statusCode: 500);
    }
}
```

*Figure 14 - Using the cameraId received from the query parameter, it creates a camera of type ENTRANCE\_LPR, makes the detectLicensePlate call, saves the result to the database, and returns DetectionResultDto as JSON if successful; in case of errors, it provides an error message with the appropriate HTTP code.*

gRPC protocol is preferred for the data transmission between Java-based service layer and Python-based YOLO unit. The primary reason behind using gRPC is transferring dense image data and detection results between systems in milliseconds. It provides data transmission in binary format, small size and with high speed due to the fact that gRPC uses HTTP/2 protocol and protocol buffers (Protobuf). It has also a special feature that is bidirectional streaming, and this feature provides high performance for real-time parking data flow. On the other hand, use of gRPC requires additional operational complexities such as defining .proto files and cross-language code generation. Nevertheless, this complexity has been accepted as an engineering trade-off in order to meet the low latency requirement that the system needs.

```

try {
    LicensePlateRequest request = LicensePlateRequest.newBuilder()
        .setCameraId(camera.getCameraId())
        .setImageData(ByteString.EMPTY)
        .build();

    LicensePlateResponse response = blockingStub.detectLicensePlate(request);

    String plateText = response.getLicensePlateText();
    lastConfidence = response.getConfidence();

    return (plateText == null || plateText.isEmpty()) ? null : plateText;
} catch (Exception e) {
    System.err.println("gRPC license plate detection failed: " + e.getMessage());
    throw new RuntimeException("License plate detection failed", e);
}

```

Figure 15 - It creates a `LicensePlateRequest` with the camera ID and makes a `detectLicensePlate` call via gRPC; it reads the license plate text and confidence score from the response and returns the text (null if empty).

```

syntax = "proto3";

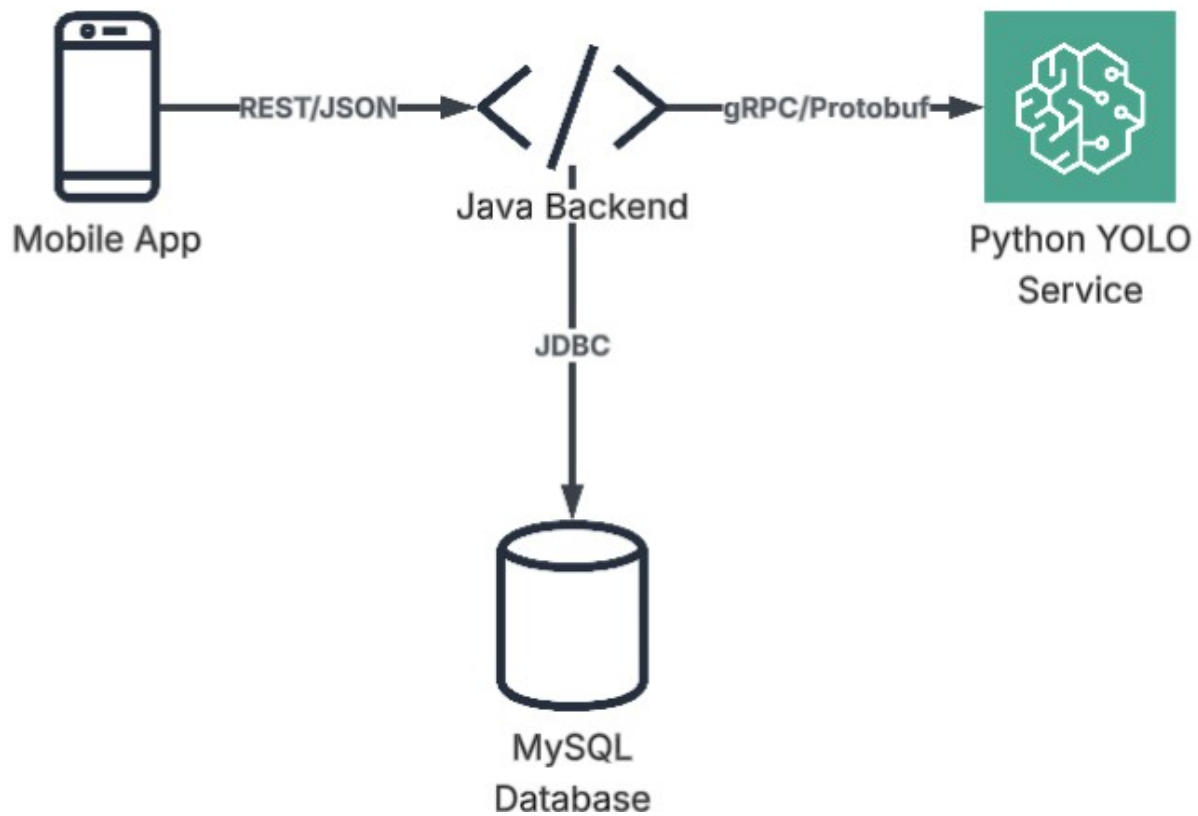
package com.parkomfy.grpc;

option java_multiple_files = true;
option java_package = "com.parkomfy.grpc";
option java_outer_classname = "DetectionProto";

message DetectionRequest {
    string camera_id = 1;
    bytes image_data = 2;
    string slot_id = 3;
}

```

Figure 16 -The `DetectionRequest` message is defined in the `com.parkomfy.grpc` packet using Proto3 syntax; it specifies the contract for gRPC requests that includes the camera ID, image data (binary), and slot ID.



*Figure 17 - The Mobile App Architecture Diagram illustrates a comprehensive system where a mobile app interfaces with a Java backend through REST/JSON, enabling interactions with a MySQL Database via JDBC and a Python YOLO Service through gRPC/Protobuf.*

## 4.3. Implementation Issues

### 4.3.1 Problem: YOLO Detection Accuracy

-Some big vehicles can block other park slot if the camera angle not good.

```
private boolean isValidGeometry(com.parkomfy.grpc.BoundingBox bbox, ParkingSlot slot) {
    // Standard parking space dimensions
    double slotWidth = 2.5;
    double slotHeight = 5.0;
    double slotArea = slotWidth * slotHeight;

    double bboxWidth = bbox.getWidth();
    double bboxHeight = bbox.getHeight();
    double bboxArea = bboxWidth * bboxHeight;

    if (bboxWidth > 0) {
        double aspectRatio = bboxHeight / bboxWidth;
        if (aspectRatio < MIN_ASPECT_RATIO || aspectRatio > MAX_ASPECT_RATIO) {
            System.out.println("[GEOMETRY FILTER] Bad camera angle: aspect=" +
                String.format("%.2f", aspectRatio) + " | slot=" + slot.getSlotId());
            return false;
        }
    }

    double sizeRatio = slotArea > 0 ? bboxArea / slotArea : 0;
    if (sizeRatio < MIN_SIZE_RATIO || sizeRatio > MAX_SIZE_RATIO) {
        System.out.println("[GEOMETRY FILTER] Partial/incorrect park: size=" +
            String.format("%.2f", sizeRatio) + " | slot=" + slot.getSlotId());
        return false;
    }

    return true; // Geometry valid: proper full parking detected
}
```

Figure 18

The aspect ratio (height/width ratio) of the bounding box in the gRPC response is used to filter out vehicles that are found to be extremely narrow or broad (indicating inadequate camera angle). Furthermore, cars that are incorrectly or partially parked are excluded according to the bounding box area to occupancy area ratio. More precise occupancy detection is made possible by the system's ability to record detections with improper geometry to the console.



### 4.3.2 Problem: License Plate Recognition Errors

- License plate recognition sometimes makes up random letter or number.

```
private String validatePlateWithMultiFrame(String plateText, double confidence) {
    // Confidence threshold check
    if (confidence < MIN_PLATE_CONFIDENCE) {
        System.out.println("[PLATE FILTER] Low confidence: " +
            String.format("%.2f", confidence) + " | require >= " + MIN_PLATE_CONFIDENCE);
        return null;
    }

    if (plateText == null || plateText.isEmpty()) {
        plateHistory.offer(null);
        if (plateHistory.size() > PLATE_FRAME_BUFFER) {
            plateHistory.poll();
        }
        System.out.println("[PLATE FILTER] Empty detection");
        return null;
    }

    // Multi-frame consistency check
    plateHistory.offer(plateText);
    if (plateHistory.size() > PLATE_FRAME_BUFFER) {
        plateHistory.poll();
    }

    if (plateHistory.size() == PLATE_FRAME_BUFFER) {
        boolean allConsistent = plateHistory.stream()
            .allMatch(p -> p != null && p.equals(plateText));

        if (!allConsistent) {
            System.out.println("[PLATE FILTER] Inconsistent across frames: " + plateHistory);
            return null;
        }
    }

    return plateText; // Validated plate
}
```

Figure 19

In order to eliminate temporary OCR errors caused by weather conditions and prevent incorrect license plate registrations, license plate detection is filtered with a confidence score of at least 80%, rejecting low-quality detections (blurred due to mud, snow, or rain). Moreover, it requires consistent license plate text across three consecutive frames.

### 4.3.3 Problem: Concurrent Session Management

- The same tool should not be detected twice; duplicate sessions should not occur.

➤ In YOLOInference Layer

```
// Duplicate detection prevention
long now = System.currentTimeMillis();
if (lastDetectedPlate != null && lastDetectedPlate.equals(plateText)) {
    long timeSinceLastDetection = now - lastPlateDetectionTime;
    if (timeSinceLastDetection < PLATE_COOLDOWN_MS) {
        System.out.println("[DUPLICATE PREVENTION] Plate " + plateText + " already detected " +
            timeSinceLastDetection + "ms ago (cooldown: " + PLATE_COOLDOWN_MS + "ms)");
        return null;
    }
}
```

Figure 20

➤ In DetectionService Layer

```
if (licensePlateText != null && !licensePlateText.isEmpty()) {
    if (isDuplicateVehicleDetection(licensePlateText)) {
        System.out.println("[DUPLICATE PREVENTION] Vehicle with plate " + licensePlateText +
            " already has active parking session; skipping duplicate detection");
        result.setVehicleDetected(vehicleDetected: false); // Mark as invalid detection
    }
}
```

Figure 21

To prevent duplicate sessions, we implemented a two-level protection mechanism:

(1) In the YOLOInference layer, we check if the detected license plate has been detected within the last 30 seconds; if the same license plate is detected again within this time window, we block it (cooldown protection).

(2) In the DetectionService layer, we query all active parking sessions in the database and check if the detected license plate has already opened a session. This two-layered defense completely prevents duplicate sessions by accepting only the first legitimate detection in case of multiple detections from the same vehicle due to camera angle changes, system latency, or fast consecutive shots, thus keeping accounting and generation data accurate.

## 4.4. Tools and Techniques Used

### 4.4.1 Backend Development (Java Spring Boot)

The `@RestController` tells Spring Boot that this is a REST controller; `@RequestMapping("/api/v1")` groups all endpoints under `/api/v1`; `@CrossOrigin(origins = "*")` allows mobile applications to send requests to this API; loose dependency is achieved by injecting `IParkingService` with Dependency Injection in the constructor; the `@GetMapping` endpoint takes the `areaId` parameter from the client, queries the `ParkingArea` from the database, populates the `ParkingStatusDto` object (total slots, available slots, occupancy rate), and returns a JSON response; in case of an error, it returns an error message with a 500 HTTP code.

```
@RestController
@RequestMapping("/api/v1")
@CrossOrigin(origins = "*")
public class ParkingApiController {

    private final IParkingService parkingService;

    public ParkingApiController(IParkingService parkingService) {
        this.parkingService = parkingService;
    }

    @GetMapping("/parking/status")
    public ApiResponse<ParkingStatusDto> getParkingStatus(@RequestParam("areaId") String areaId) {
        try {
            ParkingArea area = repository.getArea(areaId);

            ParkingStatusDto status = new ParkingStatusDto();
            status.setAreaId(area.getAreaId());
            status.setTotalSlots(area.getParkingSlots().size());
            status.setAvailableSlots(area.getAvailableSlotCount());
            status.setOccupancyRate((double) area.getOccupiedSlotCount() / area.getParkingSlots().size());

            return ApiResponse.success(status, "Status başarıyla alındı");
        } catch (Exception e) {
            return ApiResponse.error("Error: " + e.getMessage(), 500);
        }
    }
}
```

Figure 22



#### 4.4.2 AI/ML Integration (YOLO + gRPC)

LicensePlateRequest sends the camera ID and picture data, LicensePlateResponse sends the license plate text and trust score, and YOLODetectionService sets up the gRPC app that talks to the Python YOLO server. The .proto file specifies the contract for gRPC communication. The Protoc compiler turns data types into binaryProtobuf format and then utilizes that structure to make Java code snippets.

```
@Override
public String detectLicensePlateBoundingBox(Camera camera) {
    if (camera.getType() != Camera.CameraType.ENTRANCE_LPR) {
        throw new IllegalArgumentException("Camera must be ENTRANCE_LPR type for license plate detection");
    }

    if (!useGrpc) {
        // SIMULATION MODE
        if (Math.random() > 0.3) {
            lastConfidence = 0.80 + (Math.random() - 0.5) * 0.1;
            String[] simulatedPlates = {"34ABC123", "34XYZ789", "06DEF456"};
            return simulatedPlates[(int)(Math.random() * simulatedPlates.length)];
        }
        lastConfidence = 0.0;
        return null;
    }

    // GRPC MODE - Call Python YOLO service for license plate detection
    try {
        LicensePlateRequest request = LicensePlateRequest.newBuilder()
            .setCameraId(camera.getCameraId())
            .setImageData(ByteString.EMPTY)
            .build();

        LicensePlateResponse response = blockingStub.detectLicensePlate(request);

        String plateText = response.getLicensePlateText();
        lastConfidence = response.getConfidence();

        return validatePlateWithMultiFrame(plateText, lastConfidence);
    } catch (Exception e) {
        System.err.println("gRPC license plate detection failed: " + e.getMessage());
        throw new RuntimeException("License plate detection failed", e);
    }
}
```

Figure 23

After verifying the ENTRANCE\_LPR camera type, the method returns a random license plate text in simulation mode, or in gRPC mode, it sends a LicensePlateRequest to the Python YOLO service and receives the license plate text and confidence score from the LicensePlateResponse. Then, it filters out errors caused by weather conditions such as mud/snow/rain using validatePlateWithMultiFrame() and returns a validated license plate or handles the error with null.

```
message LicensePlateRequest {
  string camera_id = 1;
  bytes image_data = 2;
}

message LicensePlateResponse {
  string license_plate_text = 1;
  double confidence = 2;
  BoundingBox plate_location = 3;
}

service YOLODetectionService {
  // Detect vehicles in a frame
  rpc DetectVehicles(DetectionRequest) returns (DetectionResponse);

  // Detect license plates in a frame
  rpc DetectLicensePlate(LicensePlateRequest) returns (LicensePlateResponse);

  // Batch detection for multiple slots
  rpc DetectBatch(stream DetectionRequest) returns (stream DetectionResponse);
}
```

Figure 24

The Python YOLO service sends a LicensePlateResponse message. This message contains the license plate text and confidence score in binary Protobuf format via gRPC. On the Java side, YOLOInference.blockingStub.detectLicensePlate() directly receives this Protobuf message (without DTO conversion). Then, DetectionService processes this Protobuf data in the business logic layer. After, it converts that data into a DetectionResult Java model object, and saves it to the database via IParkingRepository. In this process, gRPC Protobuf messages eliminate the need for DTO because the binary format is already an optimized data carrier.

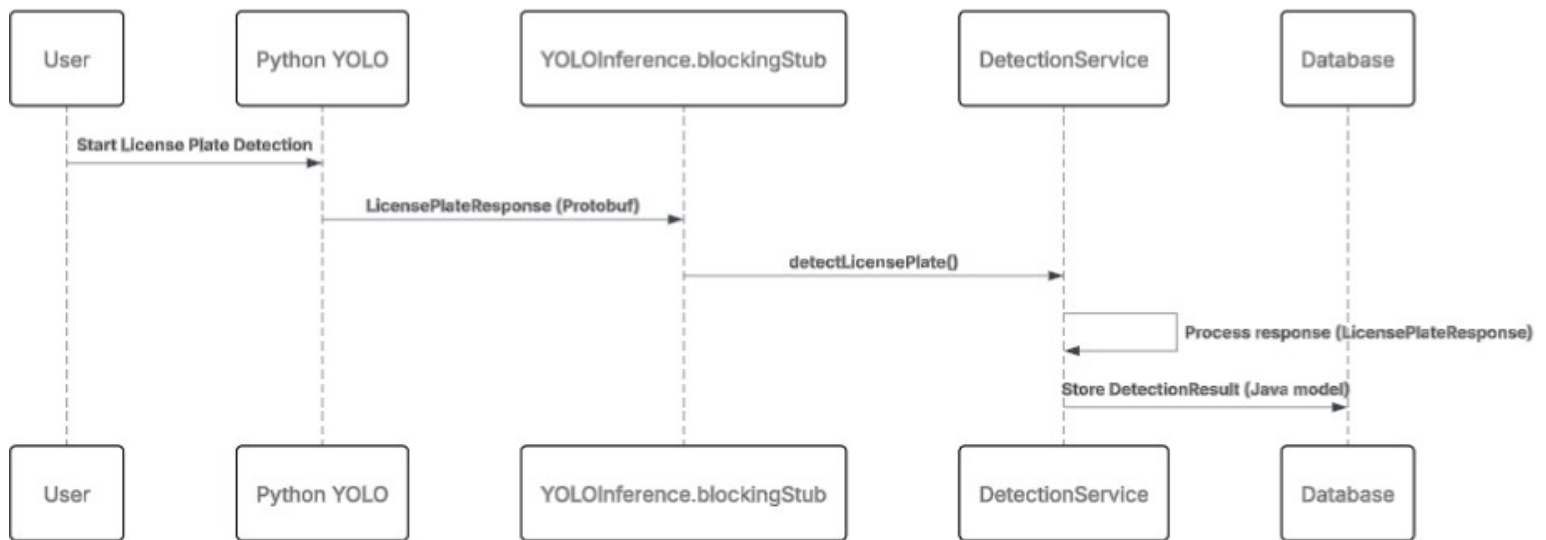


Figure 25

#### 4.4.3 Database Layer (MySQL + JDBC)

For the data layer, a sample database schema was designed and presented in the report using MySQL 8.0 (with tables such as users, vehicles, parking\_slots, parking\_sessions, payments, and pricing\_policy, along with basic relationships/foreign key connections). On the Java side, a DatabaseManager class was created in the Repository layer to correspond to this schema, and by implementing the IParkingRepository interface, a "connection point" was provided to allow the rest of the application to be developed independently of the database. However, in this version, CRUD operations (INSERT/SELECT/UPDATE, etc.) within DatabaseManager were not fully implemented, and the system's workflow was validated through mocks/simulations at this stage. Real SQL queries, transaction management, and ResultSet to model transformations will be completed in the next phase (CS402), and persistent data storage will be activated.

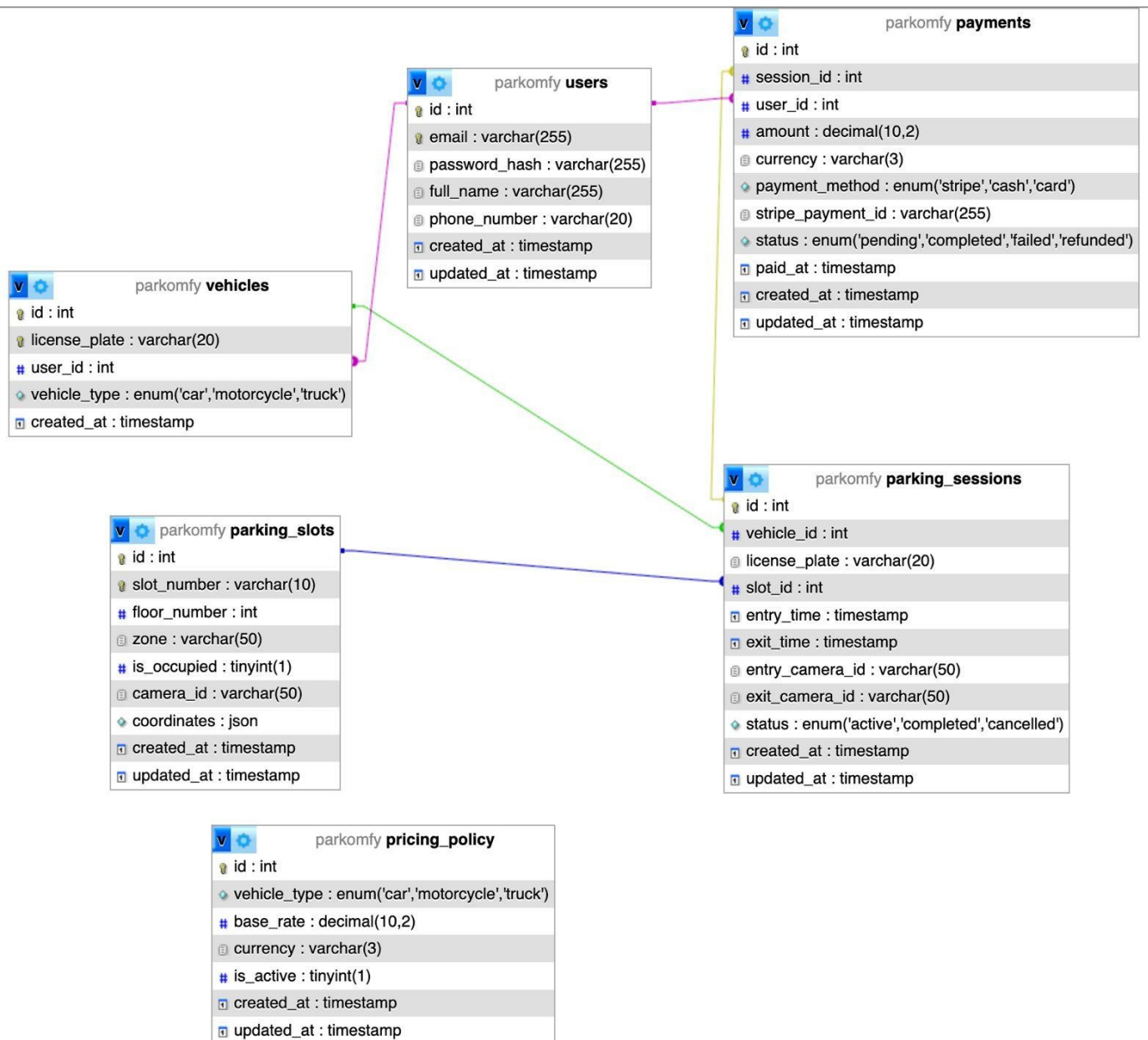


Figure 26 - This image presents a MySQL database schema showing the relationships (foreign key connections) between the user, vehicle, parking slot, parking session, payment, and pricing tables in the PARKOMFY system.

```

public DatabaseManager(String url, String username, String password) {
    this.url = url;
    this.username = username;
    this.password = password;
    initializeDatabase();
}

private void initializeDatabase() {
    try {
        // Load MySQL JDBC driver
        Class.forName("com.mysql.cj.jdbc.Driver");
        connection = DriverManager.getConnection(url, username, password);
        createTables();
    } catch (ClassNotFoundException e) {
        System.err.println("MySQL JDBC Driver not found: " + e.getMessage());
    } catch (SQLException e) {
        System.err.println("Database connection failed: " + e.getMessage());
    }
}

private void createTables() {
    // In real implementation, create all necessary tables
    // For now, this is a placeholder
    System.out.println("Database tables initialized (simulated)");
}

@Override
public void saveArea(ParkingArea area) {
    // SQL INSERT operation for parking area
    System.out.println("Saving parking area: " + area.getAreaId());
}

@Override
public ParkingArea getArea(String areaId) {
    // SQL SELECT operation
    // For now, return null (would query database)
    return null;
}

```

Figure 27 - This visualization shows the DatabaseManager class and its associated CRUD methods (for simulation purposes at this stage) defined in the repository layer, corresponding to the MySQL database schema.



#### 4.4.4 Payment Processing (Iyzico API)

This method handles payments with Iyzico. If there are no payments in the session, it creates a new Payment. It sends the paymentMethodId and amount to IyzicoIntegration. If TransactionId is returned, it completes the payment and links it to the session; if a repository exists, it saves/updates the payment and session. If TransactionId is not returned, then it fails the payment. When an error occurred, it catches the exception, logs it, and the payment fails.

```
@Override
public Payment processIyzicoPayment(ParkingSession session, PaymentMethod paymentMethod, double amount) {
    Payment payment = session.getPayment();
    if (payment == null) {
        payment = new Payment(amount, paymentMethod, session);
    }

    try {
        // Simulate Iyzico API call
        String transactionId = iyzicoIntegration.chargePayment(
            paymentMethod.getIyzicoPaymentMethodId(),
            amount
        );

        if (transactionId != null) {
            payment.markAsCompleted(transactionId);

            session.setPayment(payment);

            if (repository != null) {
                repository.savePayment(payment);
                repository.updateSession(session);
            }
        } else {
            payment.markAsFailed();
        }
    } catch (Exception e) {
        payment.markAsFailed();
        System.err.println("Payment processing failed: " + e.getMessage());
    }

    return payment;
}
```

Figure 28

## 4.5. Feasibility Analysis

**Technical Feasibility:** YOLOv8 algorithm, provides high accuracy and real time processing power. SQL based database architecture provides high speed of processing, it also holds entrance and leave times and payment records. Payment APIs such as Stripe or Iyzico allows financial transactions in secure. Wide support of open-source libraries provides us convenience to develop the project within the right framework.

**Operational Feasibility:** System can use existing camera infrastructure so it won't need any extra physical changes. Entrance and exit ticketing, and payment that needs to human power works needless in the system. This situation makes parking management fully autonomous, minimizes operational problems and reduces waiting times for users by providing smoother flow.

**Financial Feasibility:** Traditional smart parking systems needs to magnetic or ultrasonic sensor set up costs for each slot, but PARKOMFY is a software base solution so lowers the set-up costs, and these sensors might need maintenance or repair, over time it results extra cost. Our solution needs only software updates; there is no need for any extra.

## 4.6 Utilized Knowledge

**CS202 (Database Management Systems):** SQL database schemas such as vehicle entrance and exit time, license plate records are implemented with this course achievements.

**CS320 (Software Engineering):** The projects requirement analysis and structure of the system planning were developed according to this course achievements.

## 4.7. Sustainable Development Goals

**SDG 9 (Industry, innovation and infrastructure):** Instead of building new physical infrastructure, PARKOMFY digitalizes existing camera infrastructure with artificial intelligence and computer vision technologies; it offers innovative solution that increases efficiency by transforming existing security cameras to smart sensors.

**SDG 11 (Sustainable cities and communities):** A portion of urban traffic is caused by vehicles which are looking for parking spaces. PARKOMFY reduces traffic congestion by directing drivers directly to available park spaces.

**SDG 12 (Responsible consumption and production):** With autonomous license plate recognition the system prevents unnecessary paper consumption as mentioned at 4.2. This digitalizing process encourages more responsible consumption and enables operations conducted without paper.

**SDG 13 (Climate action):** Reduce on searching time for park spaces reduces also vehicles working time; this leads to reduce fossil fuel consumption and this directly reduces carbon emission and improves air quality.

## V. Results and Discussion

In this part, there are particular research findings, how to validate the result and possible bounders of the system.

### 5.1 Existing Findings and Literature Evaluation

The research yielded the following results after that research findings were done in the first stage of project and review of case studies:

**Algorithm Selection:** When similar smart parking applications were examined, it seems that YOLO architecture performs better than the other deep learning models for real-time vehicle detection.

**Data Management:** PARKOMFY needs a particular data integrity. The successful studies in literature mostly use relational databases to manage large scale parking data. Therefore, it is good to use relational databases in PARKOMFY.

### 5.2 Addressing the Problem and Validation Approach

**Middleware results (integration):** At this phase of the project, the system has a connection between the Java backend and the Python YOLO service with gRPC. It receives vehicle detection and license plate reading responses. On the other hand, the system has a connection between mobile application and Java backend with RESTful API. The REST layer returns JSON; multi-frame license plate verification, confidence threshold, and 30-second cooldown prevent duplicate sessions. In conclusion, with this workflow system ensures that end-to-end entry/exit automation works architecturally (no field metrics yet).

**Error mitigation steps:** The system has a specific geometry filter (aspect/size ratio) that is implemented in service layer and YOLOInference class. It eliminates poor camera angles and partial parking detections. Moreover, the system has the confidence threshold and multi-frame control, and they provide reduced OCR errors caused by mud/rain. This reduces the risk of "false positive" logins at the design level.



**Payment flow:** Iyzico integration works within PaymentService; transactionId is retrieved and payment status (pending/completed/failed/refunded) is managed. This means the payment infrastructure is ready to reduce waiting times at the exit point.

**Data model and schema:** MySQL schema (users, vehicles, parking\_slots, parking\_sessions, payments, pricing\_policy) and Repository interfaces are ready; DatabaseManager framework is defined. Persistent DB read/write and performance metrics will be completed in CS402.

**Validation plan:** Numerical accuracy (precision/recall) and manual counting comparisons per camera will be performed in CS402. Current status: Integration flow and error mitigation strategies are ready, but performance metrics will be collected through field testing.

### **5.3 Known Limitations**

As a result of the theoretical research done and design analysis, possible technical limitations which the system may be faced:

**Environmental Factors:** When case studies in literature are investigated, it seems that very low light levels or extreme weather conditions may affect object detection negatively.

**Visual Range Limitations:** Some vehicles may obscure their views, and this occlusion issue leads to some parking lots are not detected, so this occlusion issue may be a serious limitation.

## VI. Related Work

This section compares PARKOMFY and other Smart Parking System's approaches.

PARKOMFY's approach combines the Java Spring Boot service layer with Python YOLO via gRPC and ensures data consistency with a MySQL relational schema. The system performs real-time vehicle and license plate detection. The YOLOv8 outputs are combined with OCR, and confidence threshold. PARKOMFY provides error reduction using controls such as multi-frame consistency and geometry validation. Poor camera angles and partial parking are eliminated by these filters, and license plate deviations caused by rain and mud are reduced. Duplicate sessions are prevented in two stages: a short cooldown for the license plate and active session control in the database. Iyzico integration is used on the payment side. In this side, transaction ID, refund, and status management are kept under a single service. A hybrid model is used for communication. The model consists of two main communication channels which are ensured by REST/JSON to the mobile client and gRPC/Protobuf to the AI service. Thus, ease and speed of integration are provided together in the system. The data model is designed with foreign keys. In the model, caching, read-replica, and archiving are planned for scalability. Simulation modes are supported in development and testing in especially that phase of the project. In this way, streams can be validated without Python or MySQL. This integrated design combines the strengths of similar projects and offers real-time automation, payment security, and fault tolerance; all under one roof.

We've seen that most similar projects only use REST; however, we chose gRPC for the communication between backend and the AI. The underlying reason for this decision is the need for speed and low latency. Binary data is transported with Protobuf, the payload is reduced, and CPU and network costs are lowered in gRPC communication. Multiple calls run in parallel on the same connection with HTTP/2 multiplexing. Type safety and versioning are clearer thanks to the .proto contract. This provides a more stable pipeline for real-time YOLO outputs.

Our architecture is hybrid: REST/JSON to the mobile client, gRPC/Protobuf to Python YOLO. The YOLOInference class is the gRPC client; DetectionService orchestrates the flow. This allows us to achieve consistent response times in 30–60 FPS scenarios. Streaming support will be easier in the future. Setup is complex, so we simplified the development process with simulation mode. This choice allowed us to make a difference in performance and scalability by adopting an approach that is rarely used in literature.

## VII. Conclusion and Future Work

### 7.1 Current Status and Key Achievements

In this CS401 stage, main design plan of the system and the literature research of PARKOMFY project are completed successfully. The scope of project has been identified, availability of YOLO architecture is confirmed for object detection, and communication architecture between database of the system and mobile application is designed in this process. Also, the engineering framework is made through describing functional and non-functional requirements of PARKOMFY project.

### 7.2 Remaining Tasks and Planning

These steps in below will be followed in order to implement the project completely in the next semester:

**Data Gathering and Labeling (Ongoing):** In order to train YOLO model, creating parking dataset that includes various weather and lightning conditions.

**Model Training and Optimization (Ongoing):** Training the model for vehicle and empty lot detection, and making their validation tests.

**Developing Backend/REST/gRPC (Done):** The gRPC pipeline has been established with the Java Spring Boot backend, REST APIs, and Python YOLO; the next step is performance/scaling testing.

**System Integration and Testing:** Camera → Python YOLO → gRPC → Java backend → REST → mobile end-to-end testing; field validation (precision/recall, manual counting) will be done in CS402.

**Mobile Application (Ongoing):** The React Native interface will connect to REST endpoints, and session/payment flows will be added.

**Payment Integration (Ongoing):** The Iyzico integration will be completed with actual API calls and the refund flow.

**Database Operations (Upcoming):** DatabaseManager will be deployed with real MySQL CRUD; indexing, caching, and backup strategies will be implemented.

### 7.3 Economic and Social Factors

It is expected that fuel consumption is reduced due to minimized driver's spending time that is sought for parking lots with adoption of PARKOMFY. Therefore, it will not ensure economic savings only in individual scope but also ensure that savings in national scope. For social perspective, reduction of traffic density will lower the stress of driver and it will also increase life quality in city.

### 7.4 Global Health, Environmental and Security Factors

**Environmental and Health:** The time that vehicles seek for empty lot will be reduced, so this will reduce carbon emissions directly and benefit for the global warming challenge.

**Security and Legal Issues:** There will be camera images and there must be managed legally with respect to the KVKK, so data security will be ensured with encrypted API connections.

## 7.6 Future Work

The following features are planned to be added in order to expand the scope of this project:

**Database Integration and Data Layer:** In this project at this phase, DatabaseManager class is running in simulation mode. All CRUD operations are logging to the console. However, we aim to establish the real MySQL connections in the next CS402 phase of the project. The system will have SQL injection protection and connection pooling features successfully integrated. In addition, the transaction management of the project will be implemented and the scripts for data migration will be prepared. We aim to create separate configurations for the testing and production environments.

**YOLO Model and Python Service Integration:** The system will have a gRPC server on Python side, and the trained YOLOv8 model will be loaded. DetectVehicles and DetectLicensePlate service methods will be fully implemented. In Java side, YOLOInference class will make real gRPC calls. Especially, the special dataset that includes license plates and car types in Turkey will be gathered for model fine-tuning. Instead of simulation like in this phase, we aim to take image flow from real IP cameras.

**Mobile Application Development:** We did not start developing mobile app in this phase, however in CS402, the mobile application for iOS and Android will be developed via React Native. REST API endpoints in the backend will be used. Users will be able to view parking status. Session initiation and termination can be done via the mobile application. A push notification system will be established. Notifications will be sent when parking time is about to expire or when payment is required. Map integration will be implemented for navigation. A user-friendly UI/UX design will be created.

**Payment System and Billing:** Real Iyzico API integration will be implemented. The currently simulated chargePayment calls will be replaced with the real API. An automatic billing system will be established. Digital invoices will be sent to the user via email or SMS. The refundPayment method will be fully functional with the Iyzico API. A workflow will be defined for refund and cancellation processes. A payment history page will be created. Users will be able to view their past payments. Multi-currency support will be added. A commission management and reporting structure will be established.

**Field Testing and Validation:** Model accuracy will be measured in a real parking area. Precision and recall metrics will be calculated. True positive, false positive, true negative, and false negative counts will be recorded. Manual comparisons will be made per camera. Occupancy data generated by the system will be compared with actual vehicle counts. A/B testing will be conducted to test different camera angles and lighting conditions. Weather tests will be performed. The system's durability in challenging conditions such as rain, snow, night, and fog will be measured. Test results will be reported, and areas for improvement will be identified.

**Smart Features and Artificial Intelligence:** Occupancy prediction will be made based on historical data. Future occupancy rates will be predicted using a machine learning model. A dynamic pricing algorithm will be developed. Fee adjustments will be made according to demand intensity. An anomaly detection system will be established. Unusual parking behaviors will be detected. Very short or very long parking times will be monitored. Repeated cancellations will be flagged. A reservation system will be added. Users will be able to reserve slots in advance. Reservation cancellation and change flows will be defined.

## **Acknowledgements**

We are truly grateful to Dr. Hasan Fehmi Ateş, our adviser, for his invaluable advice, technical criticism, and their assistance during the project's planning, design, and the research phases in our sprint meetings. His advices and experiences especially in the field of computer vision shaped our project to meet academic standards.

## References

Acuna et. Al, Smart Parking System EEL 4914 | Senior Design I | Spring 2022 | Group B Senior Design 1 Documentation, 2022  
[https://www.ece.ucf.edu/seniordesign/su2022fa2022/g17/docs/SeniorDesign1\\_FinalDocumentation\\_GroupB.pdf](https://www.ece.ucf.edu/seniordesign/su2022fa2022/g17/docs/SeniorDesign1_FinalDocumentation_GroupB.pdf)

Balajithegr8, Smart-parking-system, 2023  
<https://github.com/Balajithegr8/Smart-parking-system>

Donald C. Shoup, Cruising for parking, Transport Policy, Volume 13, Issue 6, 2006, Pages 479-486, ISSN 0967-070X,  
<https://doi.org/10.1016/j.tranpol.2006.05.005>.  
(<https://www.sciencedirect.com/science/article/pii/S0967070X06000448>)

Hussain, M. (2023). YOLO-v1 to YOLO-v8, the Rise of YOLO and Its Complementary Nature toward Digital Manufacturing and Industrial Defect Detection. *Machines*, 11(7), 677.  
<https://doi.org/10.3390/machines11070677>

İbiş et. Al, Senior Design Project T2328 Parkr Analysis and Requirement Report, 2022  
<https://parkr.vercel.app/static/media/AnalysisRequirementReport.238f11b426572013337e.pdf>

soham7kulkarni, Smart-Parking-System, 2024  
<https://github.com/soham7kulkarni/Smart-Parking-System>

## Appendix

### Figures

1

```
import cv2

# Resize
image = cv2.resize(image, (640, 640))

# Contrast enhancement
clahe = cv2.createCLAHE(clipLimit=2.0)
image_enhanced = clahe.apply(image)

# Noise reduction
image = cv2.GaussianBlur(image, (5, 5), 0)
```

2

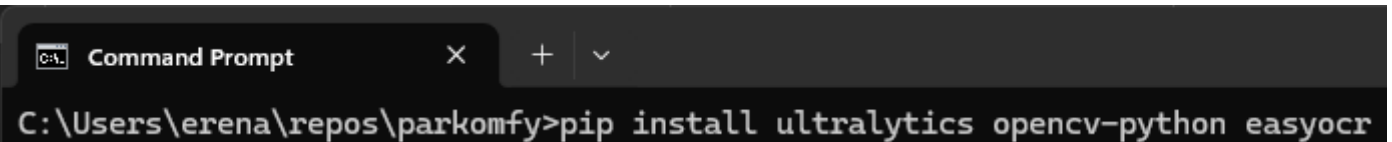
```
from ultralytics import YOLO

model = YOLO('yolov8n.pt')
```

3

```
import easyocr
reader = easyocr.Reader(['tr', 'en'])
results = reader.readtext(plate_image)
plate_text = ''.join([r[1] for r in results if r[2] > 0.5])
# Output: "34ABC123"
```

4



```
Command Prompt
C:\Users\erena\repos\parkomfy>pip install ultralytics opencv-python easyocr
```

5

```
from ultralytics import YOLO
model = YOLO('yolov8n.pt') # Pre-trained (ImageNet: 87M images)
# fine-tune with special data set(future):
# model.train(data='parking.yaml', epochs=100)
```

6

```
@SpringBootApplication
public class ParkomfySpringApplication {

    public static void main(String[] args) {
        SpringApplication.run(ParkomfySpringApplication.class, args);
    }
}
```

7

```
{
    "vehicleId": "VEH-1768043512780-268",
    "licensePlate": "34ABC123",
    "slotId": "SLOT-1A-1"
}
```

8

```
{
    "sessionId": "SESSION-1768043512816",
    "vehicleId": "VEH-1768043512780-268",
    "status": "ACTIVE",
    "entryTime": "2026-01-10T14:11:52"
}
```

9

```
{
    "sessionId": "SESSION-1768043512816",
    "status": "COMPLETED",
    "duration": "02:45",
    "fee": 25.00,
    "exitTime": "2026-01-10T17:00:00"
}
```

10

```
{
    "areaId": "AREA-001",
    "totalSlots": 20,
    "availableSlots": 12,
    "occupiedSlots": 8,
    "occupancyRate": 40.0,
    "lastUpdated": "2026-01-10T14:11:52"
}
```

11



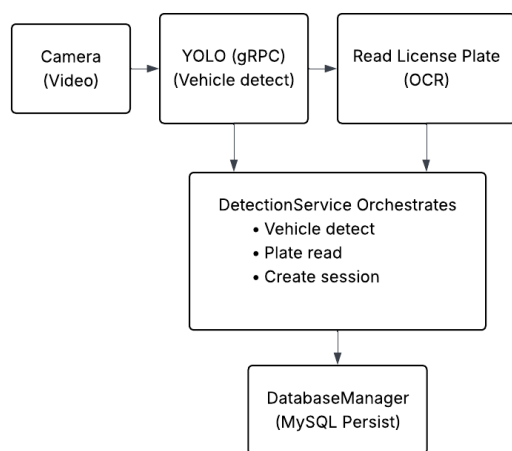
```

{
  "availableSlots": [
    { "slotId": "SLOT-1A-1", "location": "Floor 1, Zone A" },
    { "slotId": "SLOT-1A-2", "location": "Floor 1, Zone A" },
    { "slotId": "SLOT-1A-3", "location": "Floor 1, Zone A" },
    ...
  ]
}

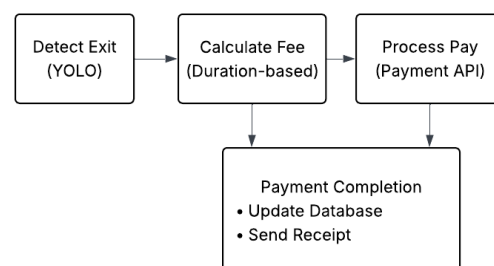
```

12

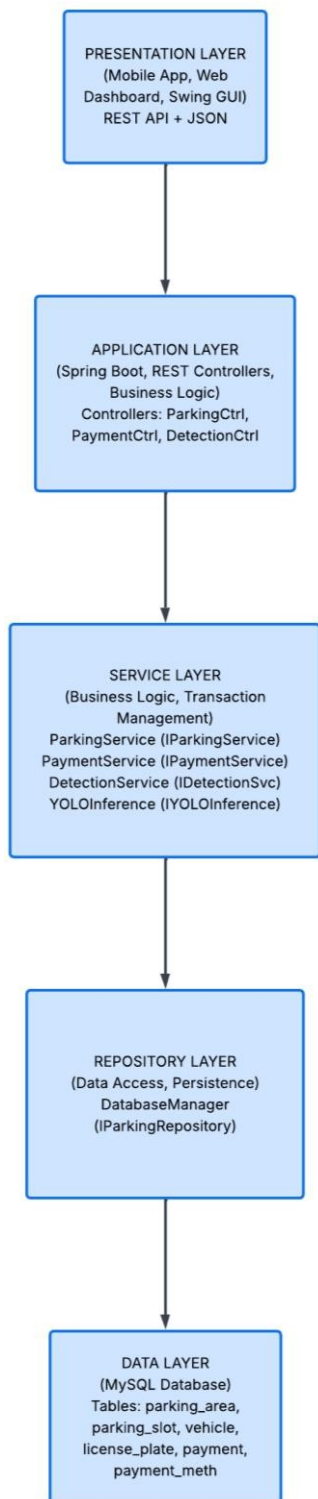
### Car Entry



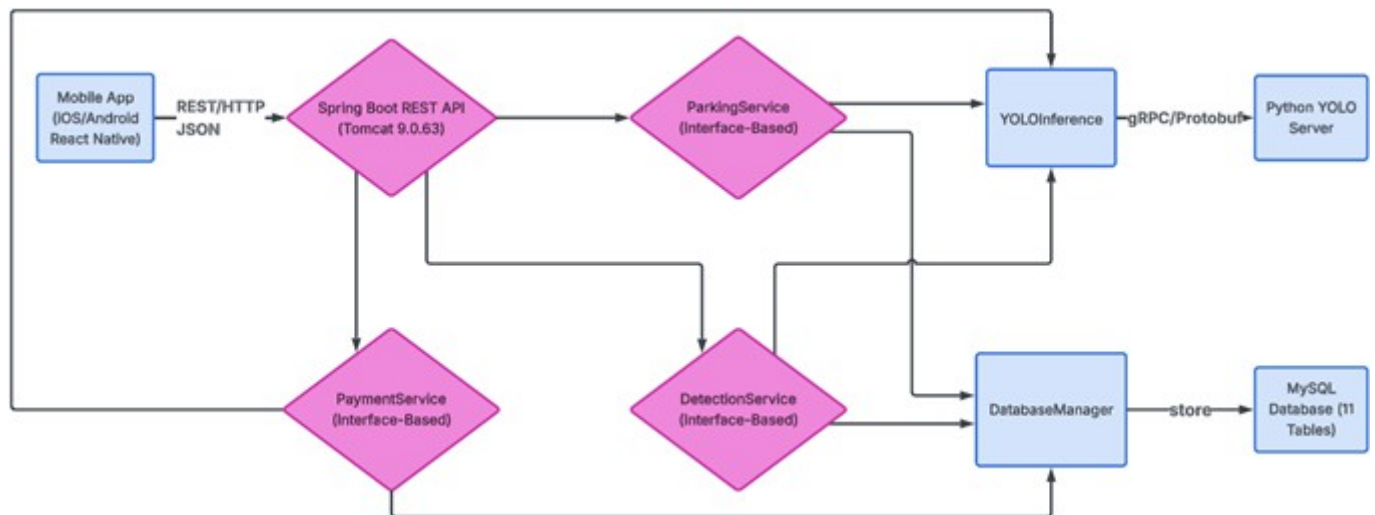
### Car Exit



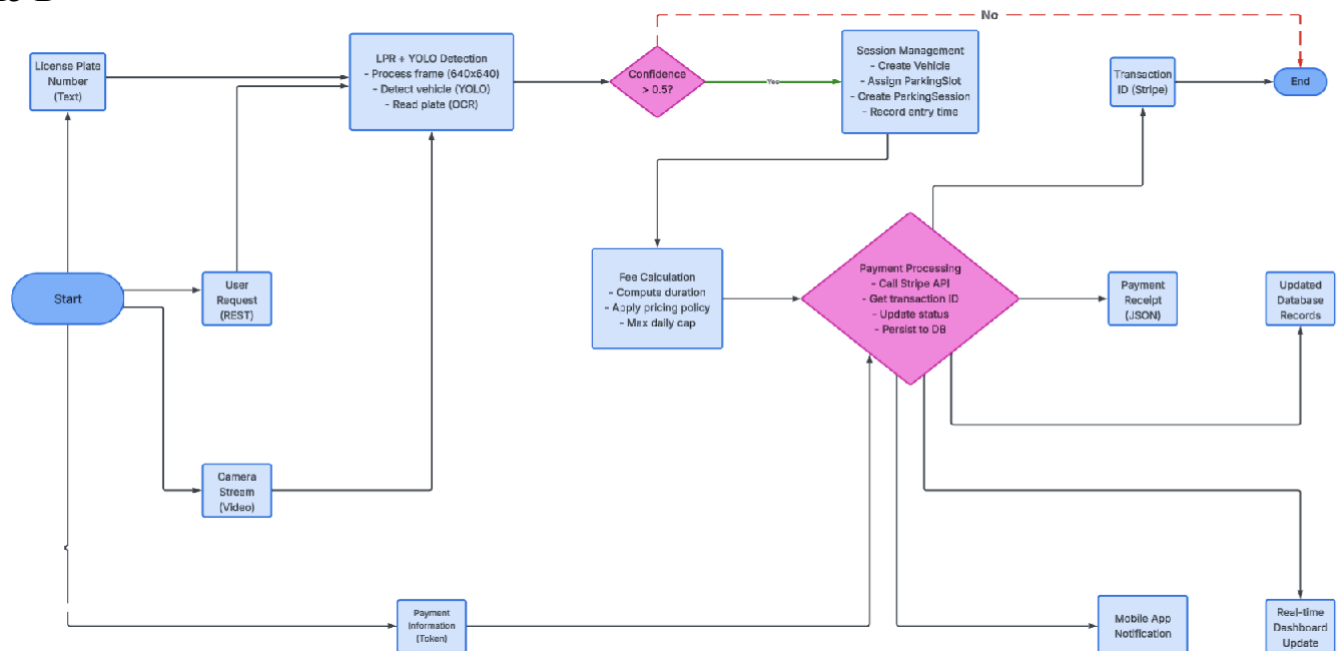
12-B



13



13-B



14

```

@GetMapping("/license-plate")
public ApiResponse<DetectionResultDto> detectLicensePlate(@RequestParam("cameraId") String cameraId) {
    try {
        Camera camera = new Camera(cameraId, "LPR-" + cameraId, Camera.CameraType.ENTRANCE_LPR, location: "Entrance");
        DetectionResult result = detectionService.detectLicensePlate(camera);
        repository.saveDetectionResult(result);
        return ApiResponse.success(new DetectionResultDto(result), message: "License plate detection executed successfully");
    } catch (IllegalArgumentException e) {
        return ApiResponse.error("Invalid camera type: " + e.getMessage(), statusCode: 400);
    } catch (Exception e) {
        return ApiResponse.error("Error executing license plate detection: " + e.getMessage(), statusCode: 500);
    }
}

```

15

```

try {
    LicensePlateRequest request = LicensePlateRequest.newBuilder()
        .setCameraId(camera.getCameraId())
        .setImageData(ByteString.EMPTY)
        .build();

    LicensePlateResponse response = blockingStub.detectLicensePlate(request);

    String plateText = response.getLicensePlateText();
    lastConfidence = response.getConfidence();

    return (plateText == null || plateText.isEmpty()) ? null : plateText;
} catch (Exception e) {
    System.err.println("gRPC license plate detection failed: " + e.getMessage());
    throw new RuntimeException("License plate detection failed", e);
}

```

16

```

syntax = "proto3";

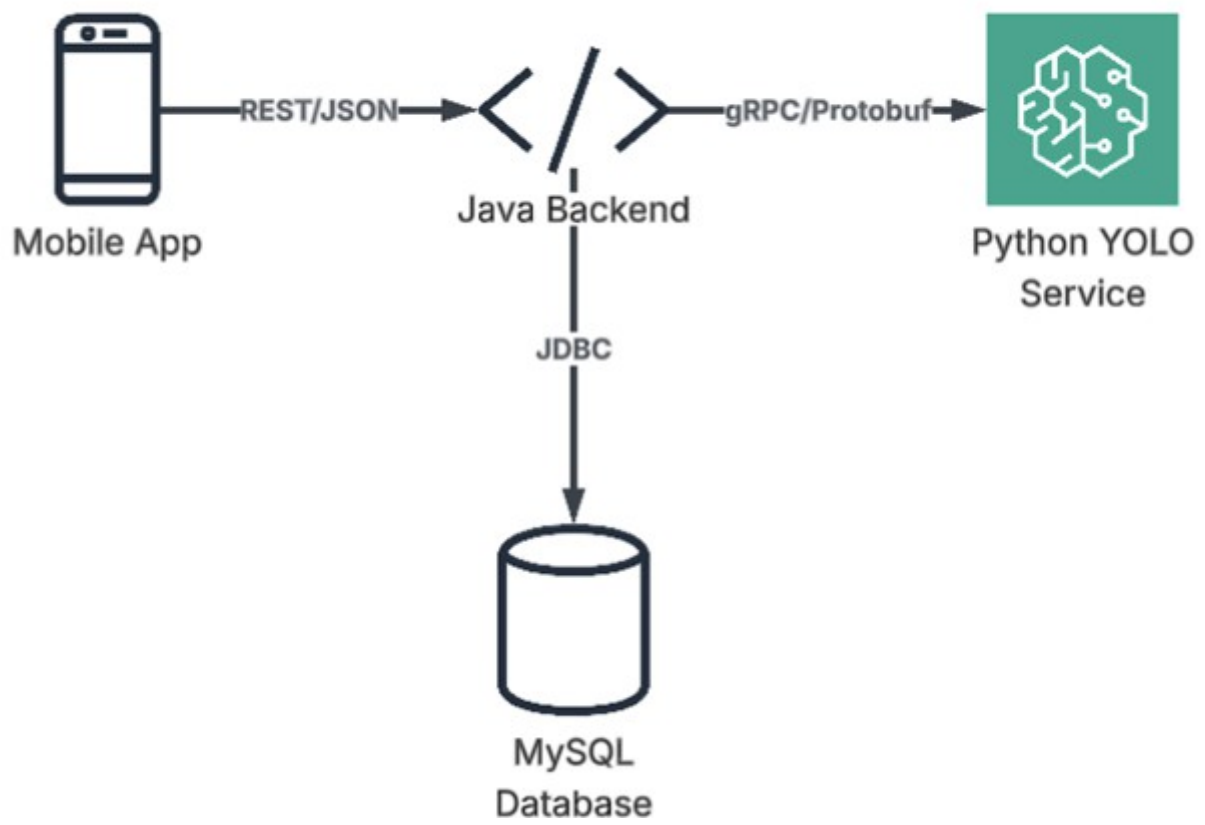
package com.parkomfy.grpc;

option java_multiple_files = true;
option java_package = "com.parkomfy.grpc";
option java_outer_classname = "DetectionProto";

message DetectionRequest {
    string camera_id = 1;
    bytes image_data = 2;
    string slot_id = 3;
}

```

17



```

private boolean isValidGeometry(com.parkomfy.grpc.BoundingBox bbox, ParkingSlot slot) {
    // Standard parking space dimensions
    double slotWidth = 2.5;
    double slotHeight = 5.0;
    double slotArea = slotWidth * slotHeight;

    double bboxWidth = bbox.getWidth();
    double bboxHeight = bbox.getHeight();
    double bboxArea = bboxWidth * bboxHeight;

    if (bboxWidth > 0) {
        double aspectRatio = bboxHeight / bboxWidth;
        if (aspectRatio < MIN_ASPECT_RATIO || aspectRatio > MAX_ASPECT_RATIO) {
            System.out.println("[GEOMETRY FILTER] Bad camera angle: aspect=" +
                String.format("%.2f", aspectRatio) + " | slot=" + slot.getSlotId());
            return false;
        }
    }

    double sizeRatio = slotArea > 0 ? bboxArea / slotArea : 0;
    if (sizeRatio < MIN_SIZE_RATIO || sizeRatio > MAX_SIZE_RATIO) {
        System.out.println("[GEOMETRY FILTER] Partial/incorrect park: size=" +
            String.format("%.2f", sizeRatio) + " | slot=" + slot.getSlotId());
        return false;
    }

    return true; // Geometry valid: proper full parking detected
}

```



```

private String validatePlateWithMultiFrame(String plateText, double confidence) {
    // Confidence threshold check
    if (confidence < MIN_PLATE_CONFIDENCE) {
        System.out.println("[PLATE FILTER] Low confidence: " +
            String.format("%.2f", confidence) + " | require >= " + MIN_PLATE_CONFIDENCE);
        return null;
    }

    if (plateText == null || plateText.isEmpty()) {
        plateHistory.offer(null);
        if (plateHistory.size() > PLATE_FRAME_BUFFER) {
            plateHistory.poll();
        }
        System.out.println("[PLATE FILTER] Empty detection");
        return null;
    }

    // Multi-frame consistency check
    plateHistory.offer(plateText);
    if (plateHistory.size() > PLATE_FRAME_BUFFER) {
        plateHistory.poll();
    }

    if (plateHistory.size() == PLATE_FRAME_BUFFER) {
        boolean allConsistent = plateHistory.stream()
            .allMatch(p -> p != null && p.equals(plateText));

        if (!allConsistent) {
            System.out.println("[PLATE FILTER] Inconsistent across frames: " + plateHistory);
            return null;
        }
    }

    return plateText; // Validated plate
}

```

```
// Duplicate detection prevention
long now = System.currentTimeMillis();
if (lastDetectedPlate != null && lastDetectedPlate.equals(plateText)) {
    long timeSinceLastDetection = now - lastPlateDetectionTime;
    if (timeSinceLastDetection < PLATE_COOLDOWN_MS) {
        System.out.println("[DUPLICATE PREVENTION] Plate " + plateText + " already detected " +
            timeSinceLastDetection + "ms ago (cooldown: " + PLATE_COOLDOWN_MS + "ms)");
        return null;
    }
}
}
```

21

```
if (licensePlateText != null && !licensePlateText.isEmpty()) {
    if (isDuplicateVehicleDetection(licensePlateText)) {
        System.out.println("[DUPLICATE PREVENTION] Vehicle with plate " + licensePlateText +
            " already has active parking session; skipping duplicate detection");
        result.setVehicleDetected(vehicleDetected: false); // Mark as invalid detection
    }
}
}
```

22

```
@RestController
@RequestMapping("/api/v1")
@CrossOrigin(origins = "**")
public class ParkingApiController {

    private final IParkingService parkingService;

    public ParkingApiController(IParkingService parkingService) {
        this.parkingService = parkingService;
    }

    @GetMapping("/parking/status")
    public ApiResponse<ParkingStatusDto> getParkingStatus(@RequestParam("areaId") String areaId) {
        try {
            ParkingArea area = repository.getArea(areaId);

            ParkingStatusDto status = new ParkingStatusDto();
            status.setAreaId(area.getAreaId());
            status.setTotalSlots(area.getParkingSlots().size());
            status.setAvailableSlots(area.getAvailableSlotCount());
            status.setOccupancyRate(((double) area.getOccupiedSlotCount() / area.getParkingSlots().size()));

            return ApiResponse.success(status, "Status başarıyla alındı");
        } catch (Exception e) {
            return ApiResponse.error("Error: " + e.getMessage(), 500);
        }
    }
}
```

23



```

@Override
public String detectLicensePlateBoundingBox(Camera camera) {
    if (camera.getType() != Camera.CameraType.ENTRANCE_LPR) {
        throw new IllegalArgumentException("Camera must be ENTRANCE_LPR type for license plate detection");
    }

    if (!useGrpc) {
        // SIMULATION MODE
        if (Math.random() > 0.3) {
            lastConfidence = 0.80 + (Math.random() - 0.5) * 0.1;
            String[] simulatedPlates = {"34ABC123", "34XYZ789", "06DEF456"};
            return simulatedPlates[(int)(Math.random() * simulatedPlates.length)];
        }
        lastConfidence = 0.0;
        return null;
    }

    // GRPC MODE - Call Python YOLO service for license plate detection
    try {
        LicensePlateRequest request = LicensePlateRequest.newBuilder()
            .setCameraId(camera.getCameraId())
            .setImageData(ByteString.EMPTY)
            .build();

        LicensePlateResponse response = blockingStub.detectLicensePlate(request);

        String plateText = response.getLicensePlateText();
        lastConfidence = response.getConfidence();

        return validatePlateWithMultiFrame(plateText, lastConfidence);
    } catch (Exception e) {
        System.err.println("gRPC license plate detection failed: " + e.getMessage());
        throw new RuntimeException("License plate detection failed", e);
    }
}

```

```

message LicensePlateRequest {
  string camera_id = 1;
  bytes image_data = 2;
}

message LicensePlateResponse {
  string license_plate_text = 1;
  double confidence = 2;
  BoundingBox plate_location = 3;
}

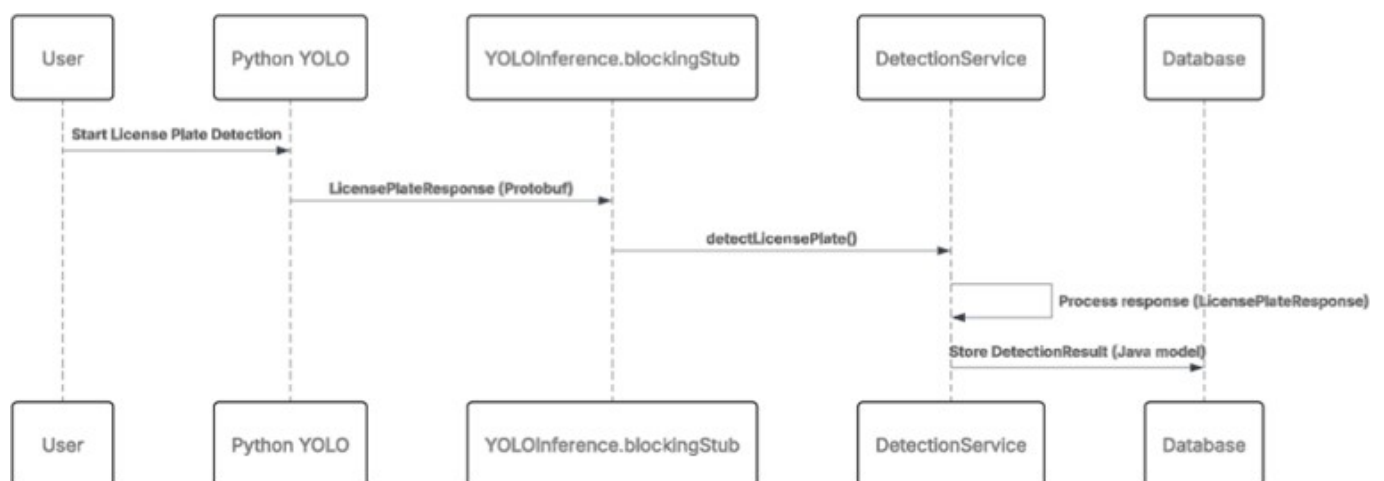
service YOLODetectionService {
  // Detect vehicles in a frame
  rpc DetectVehicles(DetectionRequest) returns (DetectionResponse);

  // Detect license plates in a frame
  rpc DetectLicensePlate(LicensePlateRequest) returns (LicensePlateResponse);

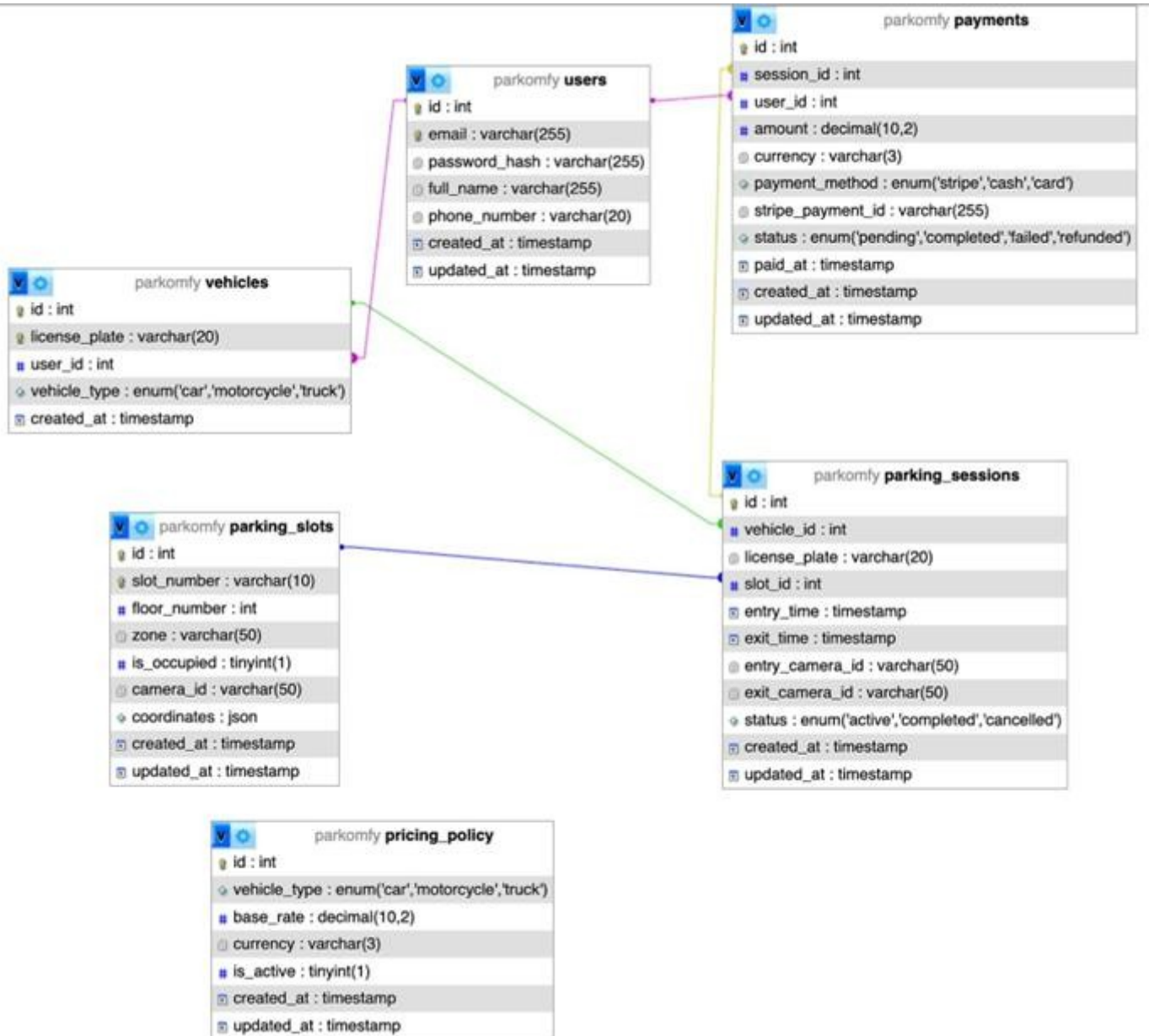
  // Batch detection for multiple slots
  rpc DetectBatch(stream DetectionRequest) returns (stream DetectionResponse);
}

```

25



26



```

public DatabaseManager(String url, String username, String password) {
    this.url = url;
    this.username = username;
    this.password = password;
    initializeDatabase();
}

private void initializeDatabase() {
    try {
        // Load MySQL JDBC driver
        Class.forName("com.mysql.cj.jdbc.Driver");
        connection = DriverManager.getConnection(url, username, password);
        createTables();
    } catch (ClassNotFoundException e) {
        System.err.println("MySQL JDBC Driver not found: " + e.getMessage());
    } catch (SQLException e) {
        System.err.println("Database connection failed: " + e.getMessage());
    }
}

private void createTables() {
    // In real implementation, create all necessary tables
    // For now, this is a placeholder
    System.out.println("Database tables initialized (simulated)");
}

@Override
public void saveArea(ParkingArea area) {
    // SQL INSERT operation for parking area
    System.out.println("Saving parking area: " + area.getAreaId());
}

@Override
public ParkingArea getArea(String areaId) {
    // SQL SELECT operation
    // For now, return null (would query database)
    return null;
}

```

```

@Override
public Payment processIyzicoPayment(ParkingSession session, PaymentMethod paymentMethod, double amount) {
    Payment payment = session.getPayment();
    if (payment == null) {
        payment = new Payment(amount, paymentMethod, session);
    }

    try {
        // Simulate Iyzico API call
        String transactionId = iyzicoIntegration.chargePayment(
            paymentMethod.getIyzicoPaymentMethodId(),
            amount
        );

        if (transactionId != null) {
            payment.markAsCompleted(transactionId);

            session.setPayment(payment);

            if (repository != null) {
                repository.savePayment(payment);
                repository.updateSession(session);
            }
        } else {
            payment.markAsFailed();
        }
    } catch (Exception e) {
        payment.markAsFailed();
        System.err.println("Payment processing failed: " + e.getMessage());
    }

    return payment;
}

```